

Informe Final de Trabajo Profesional de Ingeniería en Informática

Oxidized Neural Orchestra

Sistema distribuido de entrenamiento de modelos de aprendizaje profundo en Rust para implementar y comparar estrategias de distribución como Parameter Server y All-Reduce.

Estudiantes

Alejo Ordoñez *(Padrón 108397)*
alordonez@fi.uba.ar

Lorenzo Minervino *(Padrón 107863)*
lminervino@fi.uba.ar

Marcos Bianchi Fernández *(Padrón 108921)*
mbianchif@fi.uba.ar

Tutor: Ing. Ricardo Alfredo Veiga
rveiga@fi.uba.ar

Co-Tutor y Orientador: Dr. Ing. José Ignacio Alvarez-Hamelin
ihameli@fi.uba.ar

Facultad de Ingeniería, Universidad de Buenos Aires

Índice

Aval del director **5**

Agradecimientos **6**

Resumen **7**

Palabras clave **8**

Abstract **9**

Keywords **10**

1. Introducción **11**

2. Estado del arte **13**
2.1 Paralelismo de datos, de modelo y de pipeline 13
2.2 Parameter Server y All-Reduce 14
2.3 Strategy-Switch 14
2.4 Reducción del volumen de comunicación 14
2.5 Aprendizaje federado y sincronización diferida 15

3. Oxidized Neural Orchestra (O.N.O.) **16**
3.1 Problema detectado y/o faltante 16
3.2 Solución implementada 18
3.2.1 Componentes generales de la arquitectura 18
3.2.2 Decisiones de diseño 25

4. Simulaciones y Resultados **29**
4.1 Metodología de experimentación y validación 30
4.1.1 Pruebas automatizadas 30
4.1.2 Entorno de experimentación y reproducibilidad 30
4.2 Herramientas externas utilizadas 33
4.2.1 Bibliotecas de Rust 33
4.2.2 Bibliotecas de Python 34
4.3 Resultados 35
4.3.1 Parameter Server sincrónico con un solo servidor 35

4.3.2 Ring All-Reduce . . . 36
4.3.3 Parameter Server sincrónico con N_S servers . . . 36
4.3.4 Parameter Server asincrónico . . . 37
4.3.5 Strategy-Switch . . . 37
4.3.6 Resultados obtenidos . . . 38
4.4 Estudio derivado: impacto de los epochs offline . . . 42
4.4.1 Pregunta e hipótesis . . . 42
4.4.2 Metodología . . . 42
4.4.3 Resultados: tiempo de ejecución . . . 43
4.4.4 Resultados: exactitud . . . 45
4.4.5 Resultados: inestabilidad de la pérdida . . . 46
4.4.6 Discusión y limitaciones . . . 48
4.4.7 Conclusión del estudio . . . 48
4.5 Conclusiones del capítulo . . . 49

5. Conclusiones y Trabajo Futuro **50**
5.1 Impactos económico, sociales y ambientales del proyecto . . . 50
5.2 Trabajos futuros . . . 52
5.3 Conclusiones . . . 54

Referencias **55**

A. Aportes individuales **57**

B. Metodología de trabajo aplicada **59**
B.1 Gestión y roles . . . 59
B.2 Proceso de desarrollo . . . 59
B.3 Seguimiento, tickets y gestión del alcance . . . 60
B.4 Automatización . . . 60
B.5 Criterios de aceptación . . . 60
B.6 Artefactos e Indicadores . . . 61

C. Cronograma y riesgos del proyecto **62**
C.1 Cronograma de las actividades realizadas . . . 62
C.1.1 Fases efectivamente recorridas . . . 62
C.1.2 Contraste entre lo planificado y lo ejecutado . . . 63
C.1.3 Carga horaria efectiva . . . 63
C.2 Riesgos materializados y lecciones aprendidas . . . 64
C.2.1 Riesgos previstos en la propuesta . . . 64
C.2.2 Riesgos previstos que se materializaron . . . 65
C.2.3 Riesgos no previstos . . . 65
C.2.4 Desvíos de alcance . . . 66
C.2.5 Lecciones aprendidas . . . 66

D. Repositorio y documentación del proyecto **67**
D.1 Repositorio del proyecto . . . 67

D.2 Documentación técnica 67
D.3 Reproducibilidad de los experimentos 68

Aval del director

En Buenos Aires a los _____ días del mes de _____ del año _____ se reúnen el Ing. Ricardo Alfredo Veiga con los estudiantes de la carrera de Ingeniería en Informática: el Sr. Alejo Ordoñez (DNI 44480134), el Sr. Lorenzo Minervino (DNI 42720539) y el Sr. Marcos Bianchi Fernández (DNI 43874791). El objetivo de la reunión es revisar el Informe Final del Trabajo Profesional de Ingeniería en Informática.

Luego de haber leído el informe del Trabajo Profesional "Oxidized Neural Orchestra: sistema distribuido de entrenamiento de modelos de aprendizaje profundo en Rust para implementar y comparar estrategias de distribución como Parameter Server y All-Reduce", el Director, Ing. Ricardo Alfredo Veiga, declara que el mismo se corresponde con el trabajo realizado a lo largo del proyecto y avala el documento presentado como Informe Final, tanto en su completitud como en la claridad de redacción.

Firma y aclaración del director:

Ing. Ricardo Alfredo Veiga _____

Firmas y aclaraciones de los estudiantes:

Alejo Ordoñez _____

Lorenzo Minervino _____

Marcos Bianchi Fernández _____

Agradecimientos

A nuestro tutor, el Ing. Ricardo A. Veiga, por acompañar el trabajo de punta a punta y por sostener la exigencia sobre la forma del documento y sobre el rigor de lo que se afirma en él.

A nuestro co-tutor y orientador, el Dr. Ing. José Ignacio Alvarez-Hamelin, y al CoNexDat, Grupo de Redes Complejas y Comunicación de Datos de la Facultad de Ingeniería, por el marco en el que se desarrolló la Práctica Profesional Supervisada y por orientar las decisiones de diseño del sistema distribuido.

A nuestras familias y amigos, que bancaron los dos cuatrimestres que llevó este trabajo.

Resumen

El entrenamiento de modelos de aprendizaje profundo requiere de grandes recursos computacionales y largos tiempos de ejecución; los algoritmos que se utilizan hacen uso intensivo de CPU y GPU, y requieren de grandes capacidades de memoria para almacenar tanto los datos de entrenamiento como el modelo en sí. Junto con la revolución de la aplicación de la inteligencia artificial, minimizar ese tiempo de entrenamiento se ha vuelto un tema central, y la estrategia más escalable para lograrlo es su ejecución distribuida.

Distribuir el entrenamiento, sin embargo, no es trivial: la estrategia de distribución elegida condiciona tanto el tiempo total de ejecución como la convergencia de la optimización. En este trabajo se implementó *Oxidized Neural Orchestra* (O.N.O.), un sistema distribuido de entrenamiento en Rust que incluye una biblioteca de redes neuronales propia, construida directamente sobre la biblioteca de arreglos numéricos *ndarray* y sin recurrir a ningún framework de aprendizaje profundo existente. Sobre esa base se implementaron los tres algoritmos que sirven de referencia en el área, *Parameter Server*, *All-Reduce* en anillo y *Strategy-Switch*, junto con dos interfaces de uso: una interfaz interactiva de terminal y una biblioteca de Python construida sobre una interfaz de funciones externas.

El sistema resultante es parametrizable y permitió comparar las estrategias entre sí bajo criterios controlados. Los estudios experimentales realizados sobre MNIST muestran que los algoritmos sincrónicos convergen con buena exactitud a costa de un mayor tiempo de entrenamiento, que *Parameter Server asincrónico* es más rápido pero a costa de exactitud y que *Strategy-Switch* logra combinar las ventajas de ambos mundos exitosamente; y que postergar la sincronización entre nodos mediante epochs offline degrada la exactitud final de forma monótona sin aportar, en ese entorno, la ganancia de tiempo que la motiva. La elección de la estrategia en un despliegue real depende también de la heterogeneidad del hardware de los nodos y de la relación entre el tamaño del modelo y el ancho de banda de la red.

Palabras clave

Aprendizaje profundo, sistemas distribuidos, entrenamiento distribuido, paralelismo de datos, *Parameter Server*, *All-Reduce*, *Strategy-Switch*, Rust.

Abstract

Training deep learning models requires large computational resources and long execution times; the algorithms involved make intensive use of the CPU and the GPU, and demand large memory capacities in order to store both the training data and the model itself. Together with the revolution in the application of artificial intelligence, minimizing that training time has become a central topic, and distributing the execution across several machines is the most scalable way to achieve it.

Distributing the training, however, is not trivial: the chosen distribution strategy conditions both the total execution time and the convergence of the optimization. This work presents *Oxidized Neural Orchestra* (O.N.O.), a distributed training system written in Rust that includes a purpose-built neural network library, developed directly on top of the numerical array library *ndarray* and without relying on any existing deep learning framework. On that foundation, the three algorithms that serve as a reference in the area were implemented, *Parameter Server*, ring *All-Reduce* and *Strategy-Switch*, together with two user interfaces: an interactive terminal user interface and a Python library built on a foreign function interface.

The resulting system is parameterizable and made it possible to compare the strategies against each other under controlled fairness criteria. The experimental studies carried out on MNIST show that synchronous algorithms converge with good accuracy with a performance tradeoff, that asynchronous *Parameter Server* is faster but does not provide good convergence accuracy results, and that Strategy-Switch combines the advantages of both paradigms successfully; and that delaying synchronization between nodes through offline epochs degrades final accuracy monotonically without providing, in that environment, the time gain that motivates it. Choosing a strategy for a real deployment depends further on the heterogeneity of the node hardware and on the relation between model size and network bandwidth.

Keywords

Deep learning, distributed systems, distributed training, data parallelism, *Parameter Server*, *All-Reduce*, *Strategy-Switch*, Rust.

Capítulo 1

Introducción

El entrenamiento de modelos de aprendizaje profundo requiere de grandes recursos computacionales y largos tiempos de ejecución, y la estrategia más escalable para reducirlos es su ejecución distribuida. Sin embargo, distribuir el entrenamiento no es trivial: introduce desafíos en el diseño de la estrategia de distribución y puede degradar la convergencia de la optimización si no se realiza con cuidado. Este trabajo se propuso estudiar en profundidad las principales estrategias de entrenamiento distribuido y, a partir de dicho análisis, implementar un sistema distribuido de entrenamiento de modelos de aprendizaje profundo, con su propia biblioteca de redes neuronales, que sirviera como base para la comparación de esas estrategias y como punto de partida para el desarrollo de posibles mejoras.

El resultado es *Oxidized Neural Orchestra* (en adelante O.N.O.), un sistema distribuido escrito íntegramente en Rust que implementa tres algoritmos de entrenamiento distribuido, *Parameter Server*, *All-Reduce* en anillo y *Strategy-Switch*, sobre una infraestructura de comunicación y un motor de redes neuronales propios. Sobre ese sistema se realizaron los estudios experimentales que constituyen la validación del trabajo.

Conviene señalar desde el comienzo una particularidad de este informe. A diferencia de un trabajo cuyo objetivo es resolver un problema de negocio, aquí el producto es un *sistema de experimentación*: el valor no está solo en que el sistema funcione, sino en la evidencia que permite producir.

El presente documento se organiza de la siguiente manera. En el *Estado del arte* se relevan los avances relacionados al entrenamiento distribuido y se presentan los enfoques fundamentales que sirven de base al trabajo. El capítulo *Oxidized Neural Orchestra* reúne el *Problema detectado y/o faltante*, donde se precisa la dificultad central que se aborda, y la *Solución implementada*, que describe en detalle el sistema construido, su arquitectura y las decisiones de diseño que la explican. El capítulo *Simulaciones y Resultados* abre con la *Metodología de experimentación y validación*, sigue con las *Herramientas externas utilizadas* y el criterio con que fueron elegidas, presenta luego los *Resultados* esperados de cada algoritmo y su contraste con los obtenidos con O.N.O., suma un *Estudio derivado* sobre el impacto de los epochs offline y cierra con las *Conclusiones del capítulo*. El capítulo *Conclusiones y Trabajo Futuro* cierra el cuerpo del informe con los *Impactos económico, social y ambiental*, los *Trabajos futuros* y las *Conclusiones*. Cierran el documento las *Referencias* y los anexos, que recogen los *Aportes individuales* de cada integrante, la *Metodología de trabajo aplicada*, el *Cronograma de las actividades realizadas*, los *Riesgos materializados y lecciones aprendidas* y la

información del repositorio del proyecto y su documentación.

Capítulo 2

Estado del arte

Este capítulo aborda los enfoques de entrenamiento distribuido pertinentes para este trabajo: cómo se clasifican según qué particionan entre nodos, las dos estrategias de sincronización que sirven de base a todo lo implementado, sus combinaciones, y las técnicas de reducción de tráfico y de sincronización diferida que también se incorporaron. Este panorama sienta las bases del problema que se plantea en el capítulo siguiente.

2.1 Paralelismo de datos, de modelo y de pipeline

El entrenamiento distribuido surge como una extensión de las técnicas de paralelización en *HPC* (High Performance Computing). Ben-Nun y Hoefler (2019) y, de forma independiente, Dehghani y Yazdanparast (2023) coinciden en clasificar la concurrencia en el entrenamiento distribuido según qué se particiona entre los nodos: paralelismo de datos, que replica los parámetros en cada nodo y particiona el conjunto de datos; paralelismo de modelo, que particiona la red entre nodos y replica el lote de entrenamiento; pipelining, que asigna capas contiguas a distintos nodos y solapa sus cómputos; e híbrido, que combina los esquemas anteriores. O.N.O. se ubica exclusivamente en el primero: cada nodo mantiene una réplica completa del modelo y procesa una partición distinta del conjunto de datos, y es sobre ese esquema que se comparan las estrategias de sincronización de este trabajo.

Los primeros enfoques de paralelismo de datos en frameworks como TensorFlow (TensorFlow Developers, 2021) y PyTorch (Paszke et al., 2019), y sus implementaciones más recientes optimizadas para arquitecturas heterogéneas, reflejan la tensión constante entre escalabilidad y coherencia de los parámetros del modelo. Dehghani y Yazdanparast señalan, entre las limitaciones abiertas del área, la ausencia de benchmarks que permitan comparar el desempeño de las distintas propuestas de forma uniforme, más allá de que cada estudio suele evaluarse sobre conjuntos de datos y métricas propios (Dehghani & Yazdanparast, 2023). Esa misma ausencia es el punto de partida del problema que este trabajo aborda.

2.2 Parameter Server y All-Reduce

El equilibrio entre convergencia y eficiencia de comunicación ha motivado una amplia línea de investigación. En la actualidad, existen dos enfoques fundamentales que sirven como base para el desarrollo de nuevas técnicas: Parameter Server (M. Li et al., 2014) y All-Reduce (Z. Li, Davis, & Jarvis, 2017).

En el enfoque de Parameter Server, el modelo se divide en un conjunto de parámetros centralizados (no necesariamente en un único nodo), a los cuales los workers envían sus gradientes y desde los cuales reciben las actualizaciones, que surgen de la agregación de estas contribuciones. Este esquema admite tanto una coordinación sincrónica, en la que el servidor espera los gradientes de todos los workers antes de actualizar el modelo, como una asincrónica, en la que aplica cada gradiente apenas llega; esta última mejora la escalabilidad y reduce el tiempo de entrenamiento, aunque a costa de la coherencia entre copias del modelo. El caso extremo de esa relajación es *Hogwild!* (Niu, Recht, Ré, & Wright, 2011), que directamente abraza las *race conditions* sobre los parámetros compartidos y demuestra que, bajo la hipótesis de que cada actualización modifica solo una pequeña fracción de los parámetros (actualizaciones dispersas), la convergencia se preserva.

En contraste, en All-Reduce todos los nodos intercambian gradientes de manera directa y avanzan de forma sincrónica: en cada paso todos aplican la misma actualización, lo que preserva la coherencia entre réplicas. La formulación en anillo de Patarasuk y Yuan (2009) es óptima en ancho de banda, ya que el volumen que comunica cada nodo resulta esencialmente independiente de la cantidad de participantes, y es la que popularizó Horovod (Sergeev & Del Balso, 2018) en el ámbito del aprendizaje profundo. Trabajos posteriores como BytePS (Jiang et al., 2020) muestran que, en clústeres heterogéneos, la elección entre ambas familias deja de ser evidente.

2.3 Strategy-Switch

Uno de los claros casos de combinación de estos algoritmos es Strategy-Switch (Provatas, Chalas, Konstantinou, & Koziris, 2025), que inicia el entrenamiento de forma sincrónica sobre All-Reduce y, guiado por una regla empírica, continúa con Parameter Server asincrónico una vez que el modelo en entrenamiento se estabiliza; logrando así combinar la exactitud del régimen sincrónico de *All-Reduce* con la reducción de tiempo del régimen asincrónico de *Parameter Server*.

2.4 Reducción del volumen de comunicación

Una línea transversal a los tres enfoques es la reducción del volumen de comunicación. Aji y Heafield (2017) proponen transmitir únicamente los componentes de mayor magnitud del gradiente y acumular localmente el residuo, y Lin et al. (2018) extienden la idea con compensaciones que preservan la convergencia a tasas de compresión muy altas. Una alternativa complementaria es reducir la precisión numérica de los mensajes sin alterar la precisión del cómputo. Ambas técnicas fueron implementadas en este trabajo.

2.5 Aprendizaje federado y sincronización diferida

Finalmente, en el terreno del aprendizaje federado, McMahan et al. (2017) formalizan la idea de dejar que cada nodo entrene de forma autónoma durante varios epochs antes de sincronizar, y caracterizan el fenómeno de divergencia entre réplicas que esto induce, conocido como *client drift*. Ese mecanismo, que en O.N.O. se expone bajo el nombre de *offline epochs*, es el objeto de uno de los estudios experimentales de este informe.

Capítulo 3

Oxidized Neural Orchestra (O.N.O.)

Este capítulo presenta el sistema construido. Primero se precisa el problema concreto que motivó el trabajo, y luego se describe la solución implementada: su arquitectura, sus componentes y las decisiones de diseño que la explican.

3.1 Problema detectado y/o faltante

El entrenamiento de un modelo de aprendizaje profundo es costoso en dos dimensiones distintas: el tiempo y la memoria. Sobre conjuntos de datos grandes, un entrenamiento completo puede extenderse durante días sobre una única máquina, y tanto el conjunto de datos como el modelo y sus estados intermedios pueden exceder la memoria disponible en un solo dispositivo (Ben-Nun & Hoefler, 2019). A eso se suma una restricción práctica que define el escenario habitual: quien necesita entrenar estos modelos rara vez dispone de un equipo de gran porte, pero es frecuente que tenga varias máquinas de capacidad modesta. Alcanzar una capacidad de cómputo dada sumando el hardware que ya se tiene resulta bastante más accesible que adquirir un solo equipo que la iguale, y esa es la razón por la que se adopta el entrenamiento distribuido: reducir el tiempo repartiendo el cómputo entre varios nodos, y superar la restricción de memoria repartiendo los datos o los parámetros.

Sobre ese problema general se recorta la dificultad concreta que motivó este trabajo: **comparar las estrategias de distribución entre sí de forma controlada es difícil**. La literatura las describe y reporta resultados sobre ellas, pero las implementaciones de referencia disponibles, como las de TensorFlow (TensorFlow Developers, 2021) y PyTorch (Paszke et al., 2019), están optimizadas para escenarios de producción y no aplican el mismo trabajo de optimización de bajo nivel a cada estrategia: técnicas como el solapamiento entre cómputo y comunicación o los núcleos de cálculo especializados están más desarrolladas en unas estrategias que en otras.

TensorFlow y PyTorch son las dos bibliotecas de referencia para entrenar redes neuronales sobre múltiples nodos. Ambas ofrecen paralelismo de datos como esquema principal: cada nodo mantiene una réplica completa del modelo, procesa una partición distinta del conjunto de datos y sincroniza los gradientes resultantes con el resto antes de continuar. TensorFlow expone ese esquema a través de su API `tf.distribute.Strategy`, con variantes para entrenamiento sincrónico y asincrónico sobre una o varias máquinas; PyTorch lo hace mediante `DistributedDataParallel`, apoyado en

su biblioteca de comunicaciones colectivas `torch.distributed`. En ambos casos, quien entrena un modelo interactúa con una única llamada que envuelve el ciclo completo de sincronización, y es dentro de esa llamada donde quedan tomadas las decisiones que a este trabajo le interesa exponer y que se querría poder variar de forma independiente. Son, en concreto, cuatro.

El régimen de sincronización entre nodos. Determina si un nodo puede continuar entrenando con el gradiente que acaba de producir o si debe esperar a que los demás terminen su ronda antes de aplicarlo. Un régimen sincrónico garantiza que todos los nodos operan siempre sobre la misma versión de los parámetros, a costa de que el nodo más lento fije el ritmo de todo el clúster; un régimen asincrónico elimina esa espera, a costa de que un nodo pueda aplicar un gradiente calculado sobre parámetros que ya quedaron desactualizados. TensorFlow y PyTorch ofrecen ambos regímenes, pero como parte de la elección de la estrategia de distribución completa y no como un parámetro que se pueda variar manteniendo el resto del sistema fijo: pasar de uno a otro implica cambiar de estrategia, con su propio código de coordinación. Esa atadura entre régimen y estrategia es exactamente lo que impide aislar su efecto en una comparación.

La ubicación del optimizador dentro del ciclo de entrenamiento. Un optimizador con estado, como el descenso por gradiente con momento (Sutskever, Martens, Dahl, & Hinton, 2013) o Adam (Kingma & Ba, 2014), acumula información entre pasos, el momento, los promedios móviles, que debe residir en algún nodo. Ese estado puede vivir en cada worker, que aplica el optimizador sobre su propio gradiente antes de sincronizar, o puede vivir en un nodo central, que lo aplica una única vez sobre el gradiente ya agregado de todos los workers. La elección cambia tanto el volumen de comunicación como la semántica del entrenamiento, porque aplicar el optimizador antes o después de agregar los gradientes de distintos workers no es equivalente. En las implementaciones de referencia esta ubicación queda fija de antemano según la estrategia elegida, sin quedar disponible como variable independiente.

El esquema de particionado de los parámetros. Cuando el modelo no entra en la memoria o en el ancho de banda de un único nodo, sus parámetros deben repartirse entre varios. Partirlos por capa completa, por rango de parámetros dentro de una capa o por algún otro criterio, no es una decisión neutral: determina qué gradientes pueden calcularse y transmitirse en forma independiente y cuáles quedan acoplados entre sí. TensorFlow y PyTorch resuelven este particionado internamente, como parte de su motor de ejecución distribuida, sin documentarlo como un parámetro configurable y comparable entre estrategias.

La codificación de los mensajes que se intercambian. Los tensores y gradientes que viajan por la red pueden serializarse en un formato genérico de intercambio o transmitirse mediante una reinterpretación directa de la memoria que los contiene: el buffer de números que ya tiene el tensor se envía tal cual, en lugar de convertirlo a un formato intermedio que hay que construir y reconstruir en cada mensaje. La diferencia entre ambas opciones no es un detalle de implementación: cada copia y cada conversión de formato agrega un costo que se repite en cada ronda de sincronización, y ese costo entra a competir, en la medición final, con el que introduce el propio algoritmo de distribución. Las implementaciones de referencia fijan su formato de serialización como parte del framework, de modo que dos estrategias comparadas sobre ellas heredan siempre el mismo costo de codificación, sin que se pueda saber si ese costo favorece a una por sobre la otra.

Al comparar dos estrategias sobre dos implementaciones distintas, la diferencia medida mezcla el efecto del algoritmo con el de la calidad de su implementación, sin que exista forma de separar

ambos efectos.

Se detectó entonces la ausencia de una base común, parametrizable y de código abierto, sobre la cual las distintas estrategias de distribución compartan la misma capa de comunicación, el mismo motor de redes neuronales y el mismo plano de control; de modo que la única variable entre dos ejecuciones sea la estrategia bajo estudio. Sin esa base, cualquier comparación entre algoritmos, y en particular la evaluación de mejoras propuestas sobre ellos, queda expuesta a un sesgo que no se puede cuantificar. Construir esa base, e instrumentarla para que produzca mediciones, es el problema concreto que este trabajo resuelve.

3.2 Solución implementada

Se construyó Oxidized Neural Orchestra (O.N.O.), un sistema distribuido de entrenamiento de modelos de aprendizaje profundo. Comprende una biblioteca de redes neuronales, una capa de comunicación con protocolo propio, un plano de control que coordina la ejecución, tres algoritmos distribuidos de entrenamiento y dos interfaces de uso.

Esta sección describe la arquitectura resultante. El énfasis está puesto en las decisiones de diseño y en las razones que las motivaron, por encima de los detalles de implementación: son esas decisiones las que determinan si la base de comparación sirve a su propósito.

3.2.1 Componentes generales de la arquitectura

El siguiente diagrama resume cómo se relacionan los componentes que se describen a continuación: la capa de comunicación conecta al orchestrator con los nodos, el motor de redes neuronales corre dentro de cada nodo, y las estrategias de sincronización se apoyan en ambos.

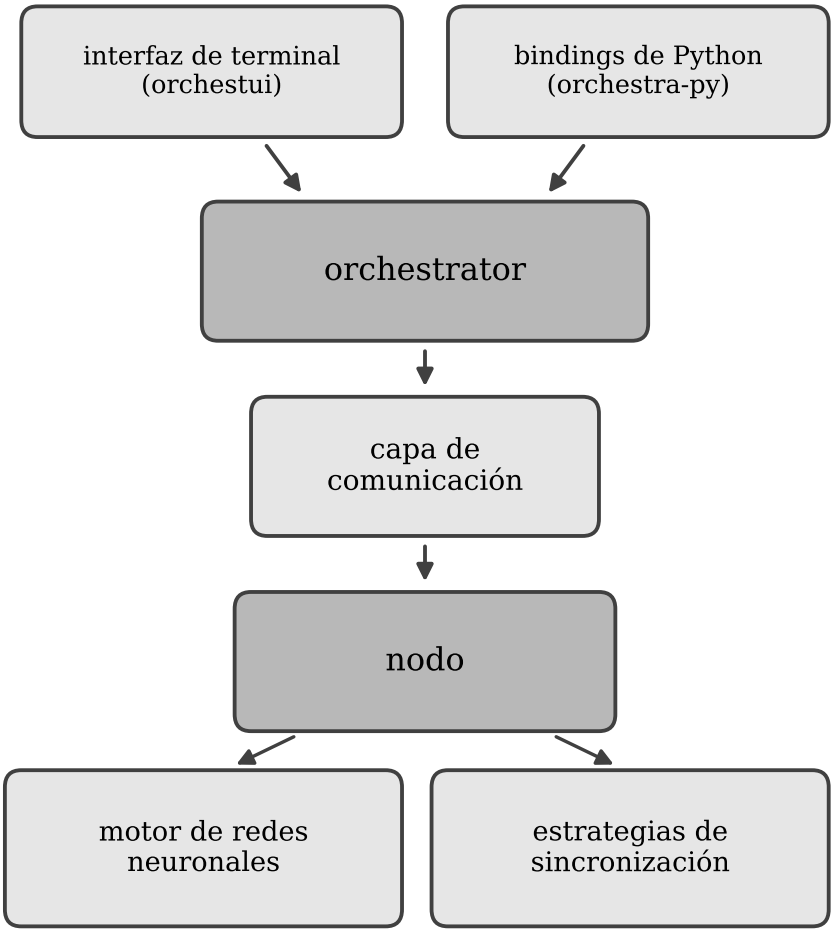


Figura 3.1: Componentes generales de la arquitectura de O.N.O. y sus relaciones.

Capa de Comunicación (`comms`). Abstrae la infraestructura de red mediante un protocolo propio. Sobre ella se apoya el handshake inicial entre nodos, un intercambio `Connect`/`Accept` en el que cada conexión declara su identificador y el rol con el que participa, y a partir del cual el extremo receptor construye el *handle* correspondiente (`WorkerHandle`, `ParamServerHandle`, entre otros) sin necesidad de conocer de antemano quién se está conectando. Un *handle* es el objeto a través del cual un nodo se comunica con otro: su tipo depende del rol del extremo remoto y solo expone los mensajes que tienen sentido para ese rol, de modo que, por ejemplo, solo el *handle* de un worker permite enviarle su partición del conjunto de datos y solo el del orchestrator permite reportarle las pérdidas de entrenamiento; cualquier otro uso es rechazado por el compilador. La capa separa además el plano de control del plano de datos: los comandos de coordinación viajan en JSON, legibles y depurables, mientras que los tensores y gradientes se transmiten mediante reinterpretación de memoria sin copias intermedias.

Motor de Redes Neuronales (`machine_learning`). Provee la biblioteca de aprendizaje profundo construida sobre el motor de arreglos numéricos `ndarray`: capas, funciones de pérdida, entrenadores y modelos, con inicializaciones de Xavier-Glorot (Glorot & Bengio, 2010) y Kaiming-He (He, Zhang,

Ren, & Sun, 2015). Los optimizadores, entre ellos el descenso por gradiente con momento (Sutskever et al., 2013) y Adam (Kingma & Ba, 2014), se definen detrás de un mismo `trait`, de modo que agregar uno nuevo no exige tocar el ciclo de entrenamiento que los invoca. Su diseño organiza el modelo como un único buffer de memoria plano sobre el cual operan las capas: cada capa no almacena sus pesos, sino solo la metadata que describe cómo interpretar su tramo del buffer, lo que permite particionar y transmitir los parámetros sin serializaciones intermedias. Los modelos se definen como una secuencia de capas (densas, convolucionales, de *max pooling*, de cambio de forma y de activación), en la que la salida de cada capa es la entrada de la siguiente. Esta es una restricción del motor y no de las estrategias de distribución: arquitecturas cuyo grafo no es una cadena, como U-Net (Ronneberger, Fischer, & Brox, 2015), una red convolucional para segmentación de imágenes que conecta cada etapa de codificación con su etapa simétrica de decodificación mediante conexiones de salto, no pueden expresarse hoy en O.N.O.

Plano de Control y Orquestación (`orchestrator`). Administra el ciclo de vida completo de un entrenamiento: arranca desde una especificación declarativa, conecta con cada nodo a través del handshake de `comms` y le asigna su rol de trabajo. Antes de comenzar, mide la latencia entre los nodos disponibles para decidir la topología, el orden del anillo en Ring All-Reduce o la ubicación de los servidores en Parameter Server. Esa medición se realiza una única vez, al inicio: cada par de nodos intercambia diez rondas de *ping* y se toma el mayor tiempo de ida y vuelta observado. La latencia real puede variar durante el entrenamiento, pero el sistema no la vuelve a medir, y la topología elegida se mantiene fija hasta el final; es una simplificación razonable en una red local estable y una limitación en redes cuya carga cambia. Durante el entrenamiento, coordina la ejecución de forma orientada a eventos, sin bloquear a la espera de cada nodo. También gestiona la detención temprana del entrenamiento, es decir, su finalización antes de agotar los epochs configurados cuando la pérdida deja de mejorar (Prechelt, 1998). Tras cada ronda de entrenamiento, cada worker reporta al orchestrator su pérdida de entrenamiento, y este registra la mayor de todas. Un contador lleva la cantidad de rondas consecutivas en las que la diferencia absoluta entre esa pérdida y la anterior no supera una tolerancia configurable; cualquier diferencia mayor lo reinicia, y cuando el contador alcanza tres rondas consecutivas el entrenamiento se detiene. Los criterios que propone Prechelt se calculan sobre un conjunto de validación; el de O.N.O. es una variante más simple, de meseta sobre la pérdida de entrenamiento, elegida por dos motivos. Primero, evita una pasada de evaluación por ronda, cuyo costo se sumaría al tiempo de entrenamiento que se quiere medir. Segundo, tomar la pérdida máxima entre workers es conservador: el entrenamiento solo se detiene cuando incluso el worker con peor pérdida dejó de mejorar. Es, además, un criterio distinto del que dispara Strategy-Switch, descrito en esta misma sección: no promedia sobre una ventana, sino que exige una sucesión ininterrumpida de mejoras despreciables.

Nodos de Cómputo (`node`). Arquitectura agnóstica de rol: todos los nodos ejecutan el mismo binario base (`node`), y es la especificación que reciben al conectarse, y no el binario que corren, lo que determina si operan como *Worker* o como *Parameter Server*. El siguiente diagrama ilustra esta asignación: el orchestrator central se conecta a un conjunto de nodos idénticos y les asigna su rol al arrancar el entrenamiento, sin que exista una imagen de despliegue distinta para cada uno.

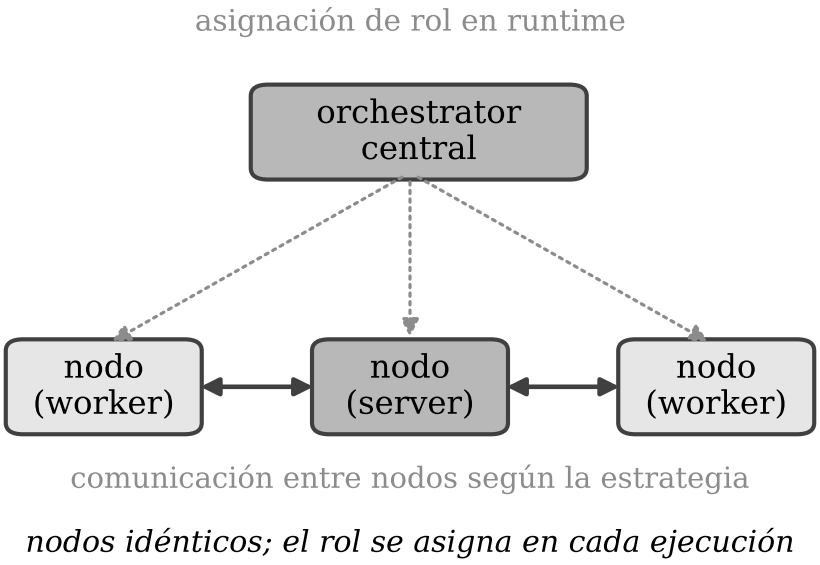


Figura 3.2: Asignación de rol en tiempo de ejecución sobre nodos idénticos.

Estrategias de Sincronización y Distribución. Soporta tres esquemas de distribución integrados, construidos sobre la misma capa de comunicación y el mismo motor de redes neuronales, de modo que la única diferencia entre ejecutarlos sea el algoritmo en sí. En lo que sigue, N_W denota la cantidad de workers que participan del entrenamiento. Los dos primeros difieren, ante todo, en su topología de comunicación: Parameter Server concentra el intercambio en uno o más servidores, mientras que Ring All-Reduce lo distribuye en un anillo sin centro, como muestra el siguiente diagrama.

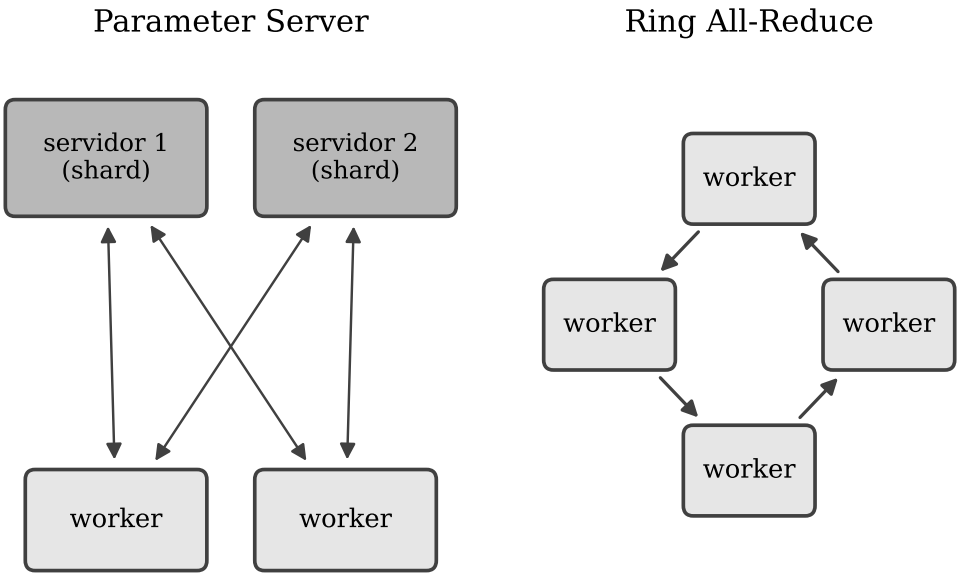


Figura 3.3: Topología de comunicación de Parameter Server y Ring All-Reduce.

Parameter Server reparte los parámetros del modelo entre uno o más servidores. En cada ronda, los workers computan localmente un gradiente sobre su partición de datos y lo empujan hacia el servidor correspondiente; el servidor lo agrega junto con el de los demás workers, aplica el optimizador con estado sobre ese gradiente agregado y devuelve los parámetros actualizados. Cuando hay más de un servidor, el particionado se realiza por capa completa: cada capa pertenece a un único servidor, y cada worker envía a cada servidor el gradiente de las capas que le corresponden. Cada servidor procesa su parte de forma independiente del resto: suma los gradientes recibidos de los N_W workers, los divide por N_W para obtener su promedio, aplica sobre ese promedio su propia instancia del optimizador y devuelve los parámetros actualizados de sus capas. Ningún servidor necesita información de otro, porque los optimizadores implementados operan elemento a elemento: la actualización de un parámetro depende solo de su propio gradiente y de su propio estado. El worker, por su parte, recompone el modelo completo a partir de los fragmentos que recibe de cada servidor. El siguiente diagrama ilustra este ciclo.

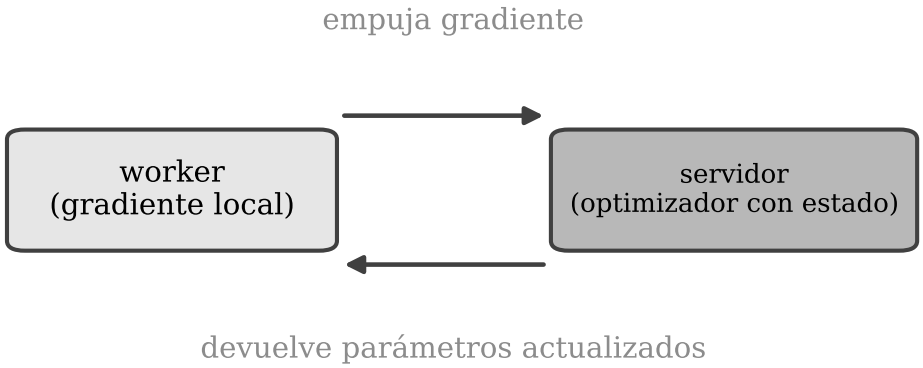


Figura 3.4: Ciclo de sincronización de Parameter Server.

Ring All-Reduce prescinde del servidor central: los N_W workers se organizan en un anillo lógico, en el que cada uno recibe mensajes solo de su anterior y envía solo a su siguiente, y reducen sus gradientes entre ellos. Para eso, cada worker divide su gradiente en N_W fragmentos de igual tamaño, y el algoritmo avanza en dos fases de $N_W - 1$ iteraciones cada una, en las que cada nodo envía a su siguiente uno de esos fragmentos, es decir, $1/N_W$ de su gradiente. En la primera fase, de *scatter* (también llamada *reduce-scatter*), cada nodo suma el fragmento que recibe con el suyo propio y reenvía el resultado; al cabo de $N_W - 1$ iteraciones, cada nodo tiene la suma completa de los N_W workers sobre uno de los fragmentos, distinto para cada nodo. En la segunda fase, de *gather* (también llamada *all-gather*), esos fragmentos ya reducidos recorren el anillo sin sumarse, de modo que, al cabo de otras $N_W - 1$ iteraciones, todos los nodos tienen los N_W fragmentos reducidos. Cada worker divide entonces la suma por N_W y obtiene el mismo gradiente promediado, que aplica de forma local (Patarasuk & Yuan, 2009). El siguiente diagrama muestra una iteración del anillo.

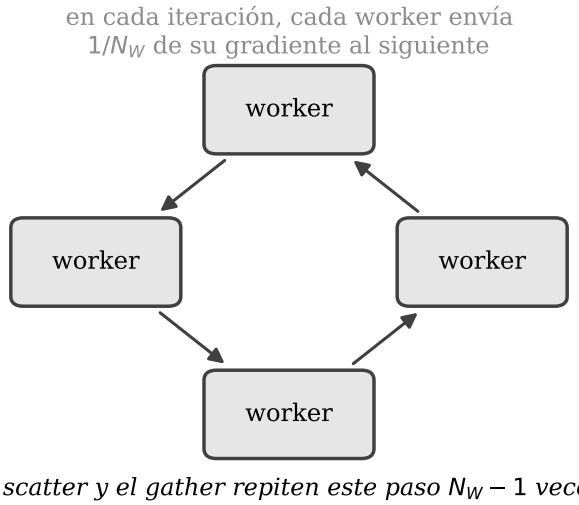


Figura 3.5: Ciclo de sincronización de Ring All-Reduce.

Strategy-Switch (Provatas et al., 2025) combina ambos esquemas: el entrenamiento arranca como All-Reduce y, cuando la pérdida se estabiliza, continúa como Parameter Server asincrónico, promoviendo a un subconjunto de workers al rol de servidor sin reiniciar el clúster. La estabilización se mide sobre una ventana con las últimas w pérdidas registradas, $L_{k-w+1}, \ldots, L_k$, mediante el promedio de su variación relativa entre rondas consecutivas,

$$s = \frac{1}{w-1} \sum_{i=k-w+2}^{k} \frac{|L_i - L_{i-1}|}{L_{i-1}},$$

y el cambio de estrategia se dispara en cuanto $s \leq 0{,}01$, es decir, cuando la pérdida varió en promedio menos del 1 % por ronda. Tanto el tamaño de la ventana, $w = 6$, como el umbral de 1 % son los del paper original, cuyos autores los eligieron de forma empírica: ventanas más grandes demoraban el cambio hasta las últimas epochs, y ventanas más chicas lo adelantaban por variaciones momentáneas de la pérdida. El paper calcula s sobre la pérdida de validación; O.N.O. lo calcula sobre la pérdida de entrenamiento, la mayor entre los workers, por las mismas razones expuestas para la detención temprana. El tamaño de la ventana w es independiente de la cantidad de workers N_W. El siguiente diagrama resume las dos fases y la condición que las separa.

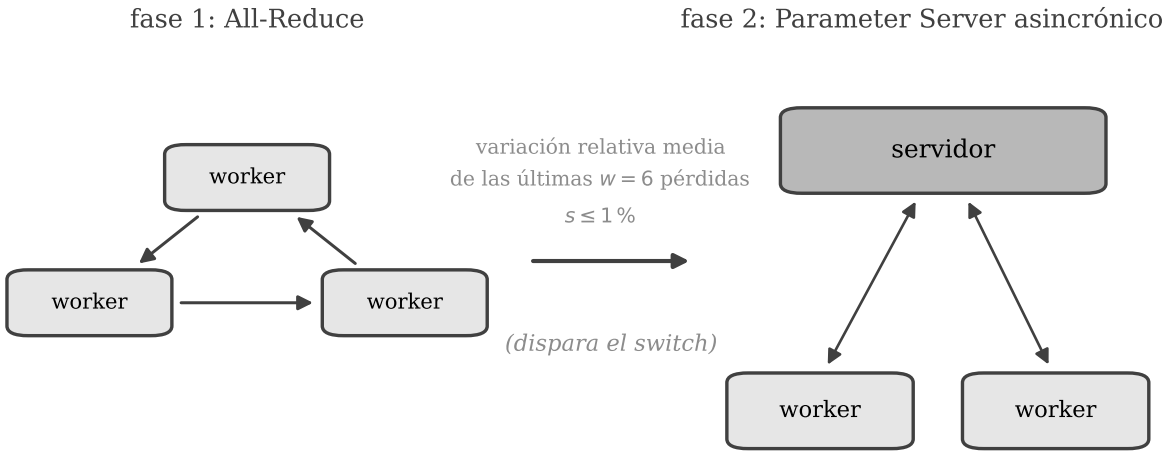


Figura 3.6: Ciclo de Strategy-Switch: de All-Reduce a Parameter Server asincrónico según la estabilización de la pérdida.

Interfaces de Usuario. Expone el sistema de dos formas complementarias: `orchestui`, una interfaz gráfica de terminal interactiva para configurar, lanzar y monitorear un entrenamiento en tiempo real, y `orchestra-py`, una biblioteca de bindings para Python construida mediante PyO3 que permite conducir el mismo sistema de forma programática, por ejemplo desde un notebook o un script de experimentación.

Tanto el motor de redes neuronales como el servidor de parámetros resuelven esta clase de variaciones mediante el mismo mecanismo: una operación se define como una implementación de un `trait` común, y el resto del sistema opera sobre el `trait` sin conocer cuál de sus implementaciones está usando en cada entrenamiento. El descenso por gradiente, el descenso con momento y Adam implementan el `trait Optimizer`; la sincronización de bloqueo por barrera y la sincronización sin bloqueos implementan el `trait Synchronizer`. Agregar una variante nueva no exige modificar el código que la invoca, solo escribir la implementación correspondiente.

3.2.2 Decisiones de diseño

Nodo agnóstico de rol (actúa sobre los Nodos de Cómputo). Todos los nodos ejecutan el mismo binario y el rol de cada uno es una especificación que el orchestrator asigna en tiempo de ejecución. Cada conexión se identifica a sí misma en el handshake inicial, un intercambio `Connect/Accept` con su identificador y su rol declarado, de modo que el otro extremo construye el handle tipado correspondiente sin necesidad de conocer de antemano quién se está conectando, como se ve a continuación. La razón principal es Strategy-Switch: conmutar de All-Reduce a Parameter Server en medio de un entrenamiento exige promover workers a servidores sin redesplegar el clúster. La misma propiedad deja abierta, como extensión, la participación de un mismo proceso en más de un entrenamiento.

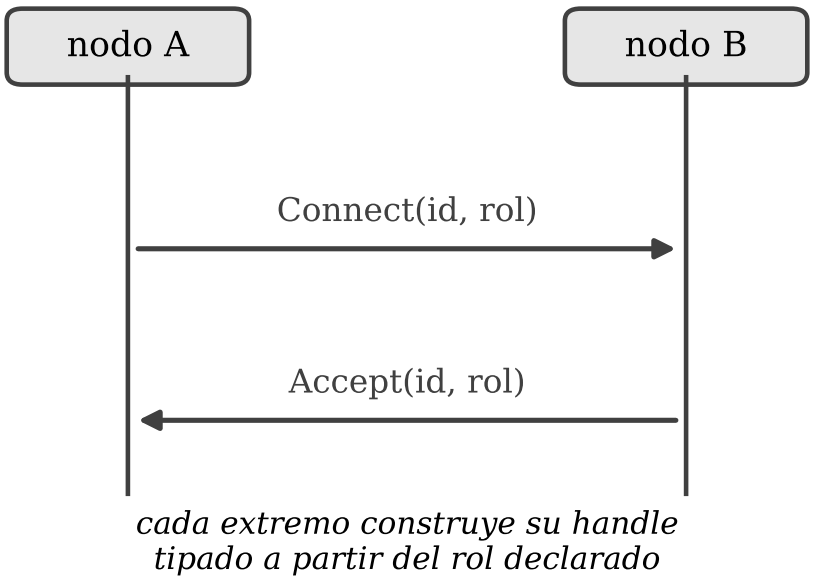


Figura 3.7: Intercambio `Connect`/`Accept` entre dos nodos durante el handshake inicial.

Modelo como buffer plano (actúa sobre el Motor de Redes Neuronales). El modelo se representa como un único buffer contiguo de memoria sobre el que las capas operan mediante vistas, en lugar de una estructura de objetos con sus pesos adentro; cada capa conserva solo la metadata que describe cómo interpretar su tramo del buffer. La alternativa, un árbol de objetos con sus propios pesos, hubiera exigido serializar y copiar cada capa por separado al particionar o transmitir el modelo, y además hubiera chocado con una restricción de Rust: al no permitir el lenguaje dos referencias mutables simultáneas sobre la misma memoria, entregar los pesos a la capa de comunicación para serializarlos sin copias hubiera forzado a acoplar el motor de redes neuronales con `comms`. Con el buffer plano, en cambio, particionar el modelo entre servidores o transmitirlo no exige copiar ni serializar los pesos: basta delimitar rangos del buffer.

Separación del plano de control y el plano de datos (actúa sobre la Capa de Comunicación). Los comandos de coordinación viajan en JSON, legible y depurable; los tensores y gradientes viajan por reinterpretación de memoria sin copias. La división responde a que ambos planos tienen necesidades opuestas: el de control es poco frecuente y conviene poder inspeccionarlo, mientras que unificarlo con el de datos hubiera obligado a elegir un único formato de por medio, perdiendo legibilidad en el control o pagando el costo de serializar cada tensor. El de datos, en cambio, concentra el volumen, y cada copia intermedia cuesta.

Compresión de gradientes con error feedback (actúa sobre la Capa de Comunicación). Antes de enviarse, cada gradiente se comprime, y esta compresión es la respuesta concreta al eje de codificación de mensajes descrito en el problema. La compresión se controla con un único parámetro, $r \in [0, 1]$, que es la fracción de componentes del gradiente que se transmiten: solo viajan los componentes cuya magnitud supera un umbral, elegido de modo que una fracción r de ellos lo supere. Ese umbral es el percentil $(1 - r)$ de las magnitudes del gradiente, y se estima por muestreo:

en vez de ordenar el gradiente completo para encontrarlo de forma exacta, se toma una muestra aleatoria de hasta 16384 componentes y se selecciona en ella el elemento de orden $k = \lfloor (1 - r)\, n \rfloor$, con n el tamaño de la muestra, mediante *quickselect*, siguiendo a Aji y Heafield (2017), en tiempo esperado lineal en lugar del tiempo log-lineal ($O(n \log n)$) que exige una ordenación completa. Los componentes descartados no se pierden: cada worker mantiene un residual persistente entre rondas que acumula lo no enviado, de modo que un gradiente pequeño pero consistente termine cruzando el umbral en una ronda posterior, la técnica de compensación que preserva la convergencia bajo compresiones altas (Lin et al., 2018). El resultado disperso se serializa como corridas de componentes contiguos que superan el umbral, no como pares índice-valor sueltos; y si esa codificación termina pesando más que enviar el gradiente completo en punto flotante de media precisión, el sistema cae automáticamente a esa alternativa densa, de modo que comprimir nunca puede resultar más caro que no hacerlo.

Particionado respetando límites de capa (actúa sobre Parameter Server, dentro de las Estrategias de Sincronización). Cuando Parameter Server fragmenta el modelo entre varios servidores, el reparto se realiza por capas enteras: ninguna capa queda dividida entre dos servidores. Es una restricción impuesta por el diseño de O.N.O. y no por el algoritmo, que admite particionados arbitrarios. La razón está del lado del worker: el motor de redes neuronales opera sobre cada capa como una vista contigua de memoria, y con el particionado por capas cada una de esas vistas apunta directamente al tramo del buffer recibido de un único servidor. Si una capa quedara repartida entre dos servidores, el worker tendría que copiar sus dos mitades a un buffer intermedio para reconstruir la capa antes de cada pasada, y deshacer esa copia al enviar el gradiente, que es justamente el costo que el buffer plano busca evitar. Del lado del servidor, en cambio, partir una capa no presentaría dificultad, porque los optimizadores operan elemento a elemento. El usuario configura únicamente la cantidad de servidores; la asignación de capas a servidores la calcula el orchestrator, que recorre las capas de mayor a menor cantidad de parámetros y asigna cada una al servidor que en ese momento tiene menos parámetros asignados, de modo que la carga de cada servidor quede balanceada. La contrapartida de esta restricción es que el balance depende de la granularidad del modelo: con pocas capas de tamaños muy dispares, o con más servidores que capas con parámetros, algún servidor queda necesariamente más cargado que el resto.

Selección de topología por latencia medida (actúa sobre el Plano de Control y Orquestación). El orchestrator mide la latencia entre todos los pares de nodos antes de entrenar y construye con esas mediciones un grafo pesado, entrada de dos problemas de optimización distintos, uno por estrategia. Para Ring All-Reduce, ordenar el anillo de forma arbitraria puede dejar dos nodos lejanos como vecinos y encarecer cada ronda de *scatter* y *gather*; el orden que minimiza la latencia total del ciclo es un circuito hamiltoniano de costo mínimo sobre el grafo medido, es decir, una instancia del problema del viajante, que el orchestrator resuelve de forma exacta por programación dinámica con memoización sobre subconjuntos de nodos, siguiendo a Held y Karp (1962), acotada a grafos de hasta `usize::BITS` vértices por representar cada subconjunto visitado como una máscara de bits. Para Parameter Server el problema es otro: elegir qué `k` nodos oficiarán de servidores de modo que ningún worker quede a una latencia desproporcionada de ellos, lo que se resuelve por backtracking sobre las combinaciones de `k` nodos, quedándose con el subconjunto de vértices más centrales: como cada worker se comunica con todos los servidores, se elige el que minimiza la mayor latencia entre cualquier worker y cualquier servidor. En ambos casos, la topología resulta de la red real y no del orden en que el usuario enumera las direcciones de los nodos en la configuración del entrenamiento,

lo que evita que una asignación arbitraria introduzca diferencias de tiempo ajenas al algoritmo bajo estudio.

Ubicación del optimizador: gradiente plano en los workers, estado después de la agregación (actúa sobre el Motor de Redes Neuronales y sobre las Estrategias de Sincronización). Durante su ronda local, cada worker recorre su partición del conjunto de datos aplicando un descenso por gradiente sin estado y acumula el gradiente resultante. El optimizador con estado configurado (momento, Adam) se aplica recién después de agregar los gradientes de todos los workers: en Parameter Server lo aplica cada servidor sobre el promedio de los gradientes de sus capas, y en All-Reduce lo aplica cada worker, de forma local, sobre el gradiente promediado que obtiene al terminar el anillo. Esta es la respuesta concreta al eje de ubicación del optimizador descrito en el problema. La razón de no aplicar el optimizador con estado antes de agregar es que su estado depende de la historia de gradientes que procesó: si cada worker aplicara Adam sobre su propio gradiente local, cada uno acumularía momentos distintos, calculados sobre particiones distintas de los datos, y las réplicas dejarían de recibir la misma actualización aunque luego se promediaran los resultados. Aplicándolo después de agregar, en cambio, todos los que lo ejecutan parten del mismo estado y reciben el mismo gradiente promediado, por lo que su estado evoluciona de forma idéntica: en All-Reduce, las N_W réplicas del optimizador se mantienen coherentes sin intercambiar nunca su estado, y en Parameter Server el estado reside solo en los servidores y no viaja por la red.

Actualización de parámetros sin bloqueos, detrás de un trait intercambiable (actúa sobre Parameter Server). El servidor de parámetros admite dos regímenes de actualización intercambiables en tiempo de configuración: uno con lock por fragmento (`BlockingStore`) y uno que directamente asume las *race conditions* al escribir sobre los parámetros compartidos, siguiendo a Niu et al. (2011) (`WildStore`). Ambos implementan el mismo `trait Store`, de modo que la elección entre corrección estricta y velocidad queda expuesta como un parámetro (`store: blocking|wild`) y no como una bifurcación de código. `BlockingStore` mantiene, además, dos buffers de gradientes por fragmento: mientras uno se está aplicando sobre los parámetros, los gradientes que llegan se acumulan en el otro, y se aplican en la actualización siguiente. De ese modo, un gradiente que llega durante una actualización nunca se pierde: el worker que lo envía no dispara una segunda actualización simultánea, y solo espera, al leer los parámetros, a que termine la escritura en curso.

Interfaz de Python como crate separado, no como feature condicional de compilación (actúa sobre las Interfaces de Usuario). La primera aproximación anotaba los tipos existentes con atributos condicionales (`#[cfg_attr(feature="python-ffi", pyclass)]`) para mantener una única fuente de verdad. Se descartó por límites duros del compilador: `#[pyclass]` no admite tipos genéricos, algunas variantes de enum no tienen una forma compatible, y ciertos tipos de error hubieran requerido implementar `Clone`. `orchestra-py` se construyó entonces como un crate aparte que envuelve al `orchestrator`, manteniendo este último en Rust puro y liberando el GIL con `py.allow_threads` durante las llamadas bloqueantes.

Estas decisiones sostienen, en conjunto, la propiedad que da sentido al sistema como base de comparación: que entre dos entrenamientos la única variable sea la estrategia de distribución.

Capítulo 4

Simulaciones y Resultados

Este capítulo pone a prueba el sistema descrito en el capítulo anterior y se organiza en cinco partes. La primera describe la *metodología de experimentación y validación*: las pruebas automatizadas que verifican que el sistema hace lo que dice hacer y el entorno simulado sobre el que se toman todas las mediciones. La segunda detalla las *herramientas externas* empleadas y el criterio con que fueron elegidas. Sobre esa base, la tercera parte, *resultados*, deriva para cada estrategia de distribución el tiempo de entrenamiento esperado a partir de un modelo analítico del costo de sincronización y lo contrasta con el medido sobre O.N.O. La cuarta es un *estudio derivado* que analiza el impacto de los epochs offline sobre la convergencia, el tiempo de comunicación y la exactitud del modelo. El capítulo cierra con las *conclusiones*, que sintetizan en qué medida las diferencias observadas entre estrategias son atribuibles al algoritmo y no a su implementación.

4.1 Metodología de experimentación y validación

Antes de presentar los resultados conviene fijar cómo se los obtuvo. Esta sección describe los dos pilares sobre los que se apoya toda medición del capítulo: las *pruebas automatizadas*, que verifican que el sistema hace lo que dice hacer, y el *entorno de experimentación*, el clúster simulado sobre el que se ejecutan tanto esas pruebas como los ensayos cuyos resultados se analizan en las Secciones 4.3 y 4.4.

4.1.1 Pruebas automatizadas

El sistema se verificó con tres tipos de pruebas. Las pruebas unitarias ejercitan cada componente de forma aislada y son las que sostienen el motor de redes neuronales: la corrección de la propagación hacia atrás no es observable a simple vista, y la única manera práctica de detectar un gradiente mal derivado es contrastarlo contra un valor calculado de forma independiente. Las pruebas de integración validan la interacción entre nodos durante una ejecución distribuida, en particular los protocolos de coordinación. Las pruebas de aceptación comprueban que cada funcionalidad requerida se comporta como fue especificada, ejecutando un entrenamiento completo sobre el entorno simulado y verificando que la convergencia obtenida sea consistente con la del entrenamiento secuencial equivalente.

Las dos primeras se escribieron con el arnés de pruebas nativo de Rust y se ejecutan mediante `cargo test`; las de aceptación se levantan sobre los entornos contenerizados descritos en la Sección 4.1.2. Las tres son automatizadas y de ejecución desatendida, y se versionan junto con el código, de modo que cada cambio pueda validarse antes de integrarse.

La suite completa se ejecuta con éxito en cada integración y su aprobación es condición obligatoria para incorporar cambios a la rama principal; en particular, las pruebas de aceptación confirman que la convergencia de las estrategias distribuidas coincide con la del entrenamiento secuencial equivalente. Esta verificación no alcanza, sin embargo, a los errores de corrección distribuida silenciosos, aquellos que no rompen ninguna prueba pero degradan la calidad del modelo; su detección motivó la incorporación de los ensayos de rendimiento y convergencia que se presentan en la Sección 4.3.

4.1.2 Entorno de experimentación y reproducibilidad

El despliegue se realiza con una única imagen de Docker, coherente con el agnosticismo de rol del sistema: no hay imagen de worker ni imagen de servidor. Un conjunto de scripts genera la configuración del clúster, ajusta la resolución de nombres y levanta el entorno a partir de un único parámetro con la cantidad de nodos.

Todas las mediciones se realizaron sobre clústeres simulados en una única máquina física: un contenedor por nodo, red *bridge* y puertos publicados. Cada contenedor se restringe a una porción fija de CPU, para emular nodos independientes, y el ancho de banda de salida de cada contenedor se limita con Pumba, que aplica sobre su interfaz de red un límite de tasa de transmisión. El capítulo reúne dos conjuntos de ensayos, ejecutados en máquinas distintas: la campaña de comparación de estrategias, cuyos resultados se presentan en la Sección 4.3, y el estudio de epochs offline de la Sección 4.4. La Tabla siguiente resume el entorno de cada uno.

Tabla 4.1: Entornos de ejecución de los dos conjuntos de ensayos del capítulo.

Característica	Comparación de estrategias	Estudio de epochs offline
Procesador	16 núcleos	Intel Core i5-8350U (4 núcleos físicos, 8 hilos lógicos; 1,7 GHz base, 3,6 GHz turbo)
Memoria	No registrada	8 GB DDR4
CPU por contenedor	1 núcleo	No registrada
Red entre nodos	10 Mbps por contenedor (Pumba)	Emulación de red hogareña: retardo, *jitter* y ancho de banda (Pumba)
Software	Docker, Pumba 1.1.7	Ubuntu 24.04 LTS, Docker 29.6.1, Rust 1.96.1, CPython 3.14.0, Pumba 1.1.7

Para la campaña de comparación de estrategias, el ancho de banda se fijó en $R = 10$ Mbps, deliberadamente bajo, de modo que la relación entre el tamaño del modelo y la velocidad de la red produzca diferencias observables entre estrategias sin recurrir a un modelo artificialmente grande.

Esa campaña se corrió sobre el conjunto de datos MNIST completo, con la arquitectura convolucional LeNet-5, de 61706 parámetros; ambos fueron introducidos por LeCun et al. (1998), que es la referencia de los dos. Como los parámetros y los gradientes viajan por la red como flotantes de 16 bits, el modelo ocupa $S \approx 1$ Mbit ($\approx 0{,}12$ MB) a los efectos de la comunicación. El entrenamiento de referencia en una sola máquina, sin distribución, tarda $T_1 = 600$ segundos. El resto de la configuración se resume en la Tabla siguiente.

Tabla 4.2: Configuración de la campaña de comparación de estrategias. Parameter Server se ejecuta siempre con un único servidor.

Parámetro	Valores
Dataset	MNIST completo
Modelo	LeNet-5
Función de pérdida	Entropía cruzada
Optimizador	Adam (Kingma & Ba, 2014) ($\alpha = 0{,}01$; $\beta_1 = 0{,}9$; $\beta_2 = 0{,}999$; $\epsilon = 10^{-8}$)
Tamaño de mini-batch	64
Epochs máximos	48
N_W	2, 4, 6, 8, 12, 16

Además de esa campaña, el repositorio incluye una suite de experimentación escrita en Python haciendo uso de la interfaz que provee el sistema para el lenguaje, y está organizada de forma declarativa en cuatro conjuntos de ensayos que miden convergencia, velocidad de ejecución, velocidad de convergencia y escalabilidad. Los resultados se persisten de forma incremental, con repeticiones en media y desvío, y la documentación se regenera automáticamente en cada corrida. Documenta

además de forma explícita sus criterios de equidad y sus fuentes de ruido: justifica el presupuesto fijo de nodos, aclara qué significa un epoch en cada ensayo, y explica por qué ciertas variantes se comparan por exactitud y no por velocidad.

4.2 Herramientas externas utilizadas

El sistema y su experimentación se apoyan en un conjunto acotado de herramientas externas. Esta sección las presenta agrupadas por rol —lenguajes, contenerización, control de versiones y emulación de red— y justifica cada elección; las bibliotecas concretas de Rust y de Python se detallan en las subsecciones finales.

Rust se eligió como lenguaje del sistema principal por varios motivos:

- Control de bajo nivel sin renunciar a garantías de seguridad.
- Buen modelo de concurrencia.
- Relevancia y crecimiento en la industria.
- Familiaridad y preferencia de los integrantes.

El primer diseño de arquitectura resolvía la concurrencia con un hilo del sistema operativo dedicado por conexión, bloqueado en una cola en memoria (un canal `mpsc`) y en cada escritura TCP; se descartó tempranamente por la complejidad que agregaba esa concurrencia embebida en el protocolo de comunicación. Migrar a un modelo async, donde la concurrencia entre conexiones se resuelve con tareas cooperativas sobre un runtime compartido en lugar de un hilo por conexión, simplificó el diseño desde ahí en adelante.

Python Se utilizó para tres propósitos:

- Interfaz de funciones externas del sistema.
- Por ser un lenguaje dominante en la industria del *Deep Learning*.
- Familiaridad con el lenguaje.

Docker Se utilizó para simular la ejecución del sistema sobre múltiples nodos y para automatizar el despliegue.

Git, GitHub, GitHub Projects y GitHub Actions Control de versiones y gestión del proceso: ramas por funcionalidad, *pull requests* con revisión obligatoria, e *issues* para el seguimiento de tickets, errores y debates de diseño. Se hizo uso de Actions para CI/CD del repositorio.

Pumba Se utilizó para simular delays de comunicación por red durante los benchmarks del sistema en entornos contenerizados de Docker. Es fácil de usar y muy parametrizable.

La totalidad de las herramientas y bibliotecas utilizadas son de código abierto, con licencias permisivas (MIT o Apache 2.0 en su mayoría). El sistema desarrollado se publica bajo licencia abierta, de modo que pueda ser retomado y extendido por terceros, según el criterio establecido en la propuesta. No se incurrió en costos de licenciamiento.

4.2.1 Bibliotecas de Rust

Estas son algunas de las bibliotecas más importantes del proyecto.

Tabla 4.3: Principales bibliotecas de Rust utilizadas en el proyecto.

Biblioteca	Uso
`tokio`	Runtime asincrónico, TCP, tareas y canales.
`ndarray`	Operaciones sobre arreglos numéricos n-dimensionales; provee la multiplicación de matrices del motor.
`rayon`	Paralelismo de datos en el almacenamiento del servidor de parámetros.
`serde` y `serde_json`	Serialización del plano de control y de los archivos de configuración, respectivamente.
`bytemuck`	Reinterpretación de memoria sin copias entre valores numéricos y bytes.
`half`	Tipo de media precisión para la compresión de gradientes.
`uuid`	Identidad estable de nodos.
`safetensors`	Exportación del modelo entrenado.
`log` y `tracing-subscriber`	Logging estructurado con filtrado por variable de entorno.
`ratatui` y `crossterm`	Interfaz interactiva de terminal.
`pyo3`	Interfaz de funciones externas con Python.
`anyhow`	Manejo de errores en la interfaz de terminal.

4.2.2 Bibliotecas de Python

Tabla 4.4: Bibliotecas de Python utilizadas en la evaluación del sistema.

Biblioteca	Uso
`maturin`	Construcción de la extensión nativa que expone el sistema a Python.
`numpy`	Lectura de los conjuntos de datos binarios en la evaluación.
`matplotlib`	Generación de todas las figuras de los experimentos.
`safetensors`	Carga de los pesos entrenados por el sistema para su evaluación.
`torch`	Línea de base de referencia.

El uso de PyTorch se restringió a benchmarks para comparar la implementación del motor de redes neuronales de O.N.O. contra una base conocida.

4.3 Resultados

El objetivo principal de los algoritmos que se implementan en este trabajo es el de reducir el tiempo de entrenamiento de un modelo de deep learning, T_1, que está dado por la suma de T_{ser} y T_{par}, los tiempos de procesamiento de las porciones de trabajo serializado y paralelizable, respectivamente. Dado que $T_{\text{ser}} \ll T_{\text{par}}$, pues, para los algoritmos que se presentan, el trabajo serializado consiste en compartir el dataset y la metadata del modelo entre los nodos, y que además T_{ser} no cambia entre los algoritmos, porque en todos ellos necesita compartirse esa información; se desprecia T_{ser} y se habla de T_{par} como T_1 en el contexto de la comparativa.

Si el costo de sincronización entre nodos fuera despreciable, el tiempo de entrenamiento sería inversamente proporcional a la cantidad de nodos P:

$$T_P = \frac{T_1}{P}.$$

En la práctica, el costo de sincronización no es despreciable; más aún, se verá más adelante que en algunos casos puede incluso crecer con la cantidad de máquinas que deben sincronizarse.

El tiempo total de entrenamiento distribuido está dado por la combinación del tiempo de ejecución local y el delay de sincronización y de inicialización:

$$T_{\text{total}} = N_{\text{epochs}} \cdot \left(\frac{T_{\text{epoch}}}{N_W} + d_{\text{sync}} \right) + d_{\text{inicialización}},$$

donde T_{total} es el tiempo total de ejecución del entrenamiento distribuido, N_{epochs} es la cantidad de épocas de entrenamiento, T_{epoch} es el tiempo de ejecución promedio de cada época, N_W es la cantidad de workers en el sistema distribuido, d_{sync} es el delay de sincronización en cada iteración y $d_{\text{inicialización}}$ es el delay de inicialización del entrenamiento distribuido.
En un entorno de computación distribuida, una pasada por el dataset en cada epoch se divide entre la cantidad de workers que las computan.

Las subsecciones que siguen derivan el delay de sincronización d_{sync} para cada estrategia. Las expresiones son de elaboración propia: se obtienen a partir de los supuestos enunciados y de la descripción de cada algoritmo en la literatura citada, y no se toman de ninguna fuente. El contraste entre esos valores esperados y los medidos sobre O.N.O. se presenta en la Sección 4.3.6.

4.3.1 Parameter Server sincrónico con un solo servidor

En Parameter Server (M. Li et al., 2014) con un solo server, el delay que introduce la sincronización del sistema está dado por el tiempo de comunicación de la ida de los parámetros del server hacia los workers, y de la vuelta de los gradientes de los workers hacia el server.

Suponiendo un mismo bandwidth R para todos los nodos, delays de encolado, propagación y procesamiento despreciables; y un modelo de tamaño S ($S = N_{\text{params}} \times 2$ bytes, dado que los parámetros se serializan como flotantes de 16 bits); el tiempo que tarda la ida de los parámetros es el de su envío en N_W mensajes unicast $N_W \frac{S}{R}$.
Suponiendo un ambiente homogéneo, en el que el tiempo de procesamiento de los workers es similar, todos comienzan a contestar con los respectivos gradientes en simultáneo. Sin embargo, el server

solo tiene una tarjeta de red con la cual recibir estos gradientes, por lo que el tiempo que tarda la vuelta de los gradientes es también $N_W \frac{S}{R}$.

El delay de sincronización $d_{\text{sync}}^{\text{PS}}$ de Parameter Server sincrónico con un solo servidor es entonces

$$d_{\text{sync}}^{\text{PS}} = 2N_W \frac{S}{R}.$$

Nótese que el delay crece con la cantidad de workers.

Cabe destacar que generalmente uno querría que el nodo que ejecuta el parameter server ejecute también un worker, dado que de otra forma se estaría desperdiciando su CPU siendo que la entidad del server está limitada por I/O. En cuyo caso el delay de comunicación entre el parameter server y el worker que residen en el mismo nodo se anula, y por tanto $d_{\text{sync}}^{\text{PS}} = 2(N_W - 1)\frac{S}{R}$.

4.3.2 Ring All-Reduce

El delay de sincronización de Ring All-Reduce (Patarasuk & Yuan, 2009) está dado por el delay del *scatter* y por el delay del *gather*. Ambas etapas del algoritmo son de $N_W - 1$ iteraciones. En ambas, la sincronización en una iteración consiste en enviar una N_W-ésima parte del gradiente del i-ésimo al $i + 1$-ésimo nodo. Dado que se desprecian los delays de encolado, propagación y procesamiento; y que las tarjetas de red son *full-duplex* (tienen canales dedicados para leer y escribir); puede pensarse que la comunicación en anillo se realiza en paralelo.

Considerando los mismos supuestos de delays de comunicación por red que se usaron para el desarrollo anterior, el delay de sincronización de All-Reduce, está dado por hacer dar vuelta una N_W-ésima parte del gradiente dos veces. Esto es

$$\begin{aligned} d_{\text{sync}}^{\text{AR}} &= 2(N_W - 1)\frac{S/N_W}{R} \\ &= 2(1 - \frac{1}{N_W})\frac{S}{R}. \end{aligned}$$

A diferencia de Parameter Server, el factor afectado por la cantidad de workers $\frac{1}{N_W} \to 0$ cuando $N_W \to \infty$,

$$\lim_{N_W \to \infty} d_{\text{sync}}^{\text{AR}} = 2\frac{S}{R}.$$

El delay de sincronización está acotado por el doble del delay de transmisión del tamaño de los parámetros del modelo.

4.3.3 Parameter Server sincrónico con N_S servers

Como ya se vio, la mayor parte del delay de sincronización en Parameter Server viene del hecho de que todos los workers tienen que comunicarse con un único servidor, y dado que este servidor puede enviar y recibir una sola tira de parámetros/gradientes a la vez, el delay escala con N_W.
Una forma de resolver este problema es incrementar la cantidad de servidores. Cada servidor tiene una porción disjunta de los parámetros del modelo, y los workers solicitan parámetros y envían

gradientes al servidor que corresponda.

En Parameter Server, por tratarse de comunicación de uno a muchos, el cuello de botella de la sincronización está del lado del servidor. Con un solo servidor ya se vio que el envío de parámetros y la recepción de gradientes ambos tardan $N_W \frac{S}{R}$, que es el máximo entre el delay desde el punto de vista de un worker y desde el del server: $\max\left(\frac{S}{R}, N_W \frac{S}{R}\right)$. Pero con N_S servers, el delay del lado del servidor se transforma en $N_W \frac{S/N_S}{R}$. Por lo tanto

$$d_{\text{sync}}^{\text{Multi-PS}} = 2\frac{N_W}{N_S}\frac{S}{R} \qquad (N_S \leq N_W).$$

Nótese que cuando $N_S = N_W$, $d_{\text{sync}}^{\text{Multi-PS}} = 2\frac{S}{R}$, que es el mismo resultado obtenido que para All-Reduce cuando N_W crece indefinidamente.

Vale la misma aclaración que se hizo en el caso para un solo servidor con respecto a que los nodos que ejecutan un server normalmente van a estar también ejecutando un worker, por lo que los delays de transmisión se anulan en esos nodos: $d_{\text{sync}}^{\text{Multi-PS}} = 2\frac{N_W-1}{N_S}\frac{S}{R}$.

4.3.4 Parameter Server asincrónico

El delay de sincronización desarrollado en los puntos anteriores se da luego de cada ronda de epochs de *todos* los workers. Esto es, cada worker computa una epoch para luego sincronizar sus parámetros con el resto, o más de una si se configuran epochs offline. Esto último reduce la cantidad de sincronizaciones, y por lo tanto solo resulta deseable cuando el costo de comunicación domina sobre el de cómputo, por ejemplo con modelos grandes o redes lentas, y a costa de una mayor divergencia entre réplicas, como se analiza en la Sección 4.4.

Parameter Server asincrónico libera al entrenamiento distribuido de esta restricción, dejando que el parameter server mueva los parámetros del modelo a medida que van llegando gradientes de los workers *sin necesidad de esperar al resto*. En este caso el costo de sincronización se elimina porque la sincronización simplemente desaparece, aunque aún se tiene que considerar el delay de transmisión $2\frac{S}{R}$ de cada worker.
Si un gradiente llega mientras el servidor todavía está aplicando el anterior, no se descarta ni dispara una actualización simultánea: en el almacenamiento con bloqueo por fragmento (`BlockingStore`), el servidor mantiene dos buffers de gradientes, de modo que el que llega se acumula en el buffer libre y se aplica en la actualización siguiente, junto con los demás que lleguen en ese intervalo. Si las llegadas son más frecuentes que lo que tarda una actualización, el efecto es que varias contribuciones se agregan en un mismo paso del optimizador, y no que se pierdan.

El problema con esta estrategia es que la convergencia empeora mucho cuando la desincronización es grande, es decir, cuando los workers mueven demasiado los pesos en la dirección sesgada que les dicta su partición de los datos; fenómeno conocido como *client drift* (McMahan et al., 2017).

4.3.5 Strategy-Switch

Strategy-Switch (Provatas et al., 2025) combina la convergencia de All-Reduce y la velocidad de ejecución de Parameter Server asincrónico. La idea es encontrar un buen mínimo local con el primer

algoritmo para luego seguir bajándolo por medio del segundo.
Considerando que d_{sync} es el tiempo en el que los nodos permanecen sin realizar ningún cómputo, ni comunicarse directamente con otro nodo, por esperar que termine la sincronización, puede decirse entonces que luego del switch ese tiempo puede llegar a anularse: su cota inferior es cero, que corresponde al caso en que ningún worker encuentra al servidor ocupado. En el peor de los casos, cuando todos los nodos se comunican con el parameter server en simultáneo, el delay llega a $d_{\text{sync}}^{\text{PS}}$.

El delay de sincronización de Strategy-Switch depende entonces de si se realizó el *switch*:

$$d_{\text{sync}}^{\text{SS}} = \begin{cases} 2(1 - \frac{1}{N_W})\frac{S}{R}, & \text{antes del switch (All-Reduce)} \\ d_{\text{sync}}^{\text{Async PS}}, & \text{después (Parameter Server asincrónico).} \end{cases}$$

con $0 \leq d_{\text{sync}}^{\text{Async PS}} \leq d_{\text{sync}}^{\text{PS}}$.

4.3.6 Resultados obtenidos

Los resultados se obtuvieron sobre MNIST (LeCun et al., 1998), con la arquitectura, los hiperparámetros y el barrido de workers detallados en la *Metodología de experimentación y validación*, y sobre el entorno simulado descrito en esa misma sección.

Un factor clave en el delay de sincronización es el tamaño del modelo S frente al bandwidth R de la red, es decir, el delay de transmisión d_{trans}. Como el tamaño de los modelos que resultan útiles para aprender MNIST no basta para que ese cociente produzca diferencias observables entre estrategias, se fijó $R = 10$ Mbps, deliberadamente muy inferior al de una red real, en lugar de recurrir a un modelo innecesariamente grande o de cambiar el problema por uno que así lo requiera. Con un modelo de $S \approx 1$ Mbit ($\approx 0{,}12$ MB) y ese bandwidth, el delay de transmisión es $d_{\text{trans}} \approx 0{,}1$ segundos.

El objetivo de un algoritmo de entrenamiento distribuido es lograr una ganancia en performance que justifique el costo de ejecución en un ambiente multi máquina, garantizando al mismo tiempo que el modelo converja; de lo contrario, el entrenamiento en sí pierde todo sentido.

Se muestra primero la comparativa de resultados de los algoritmos que operan de manera sincrónica.

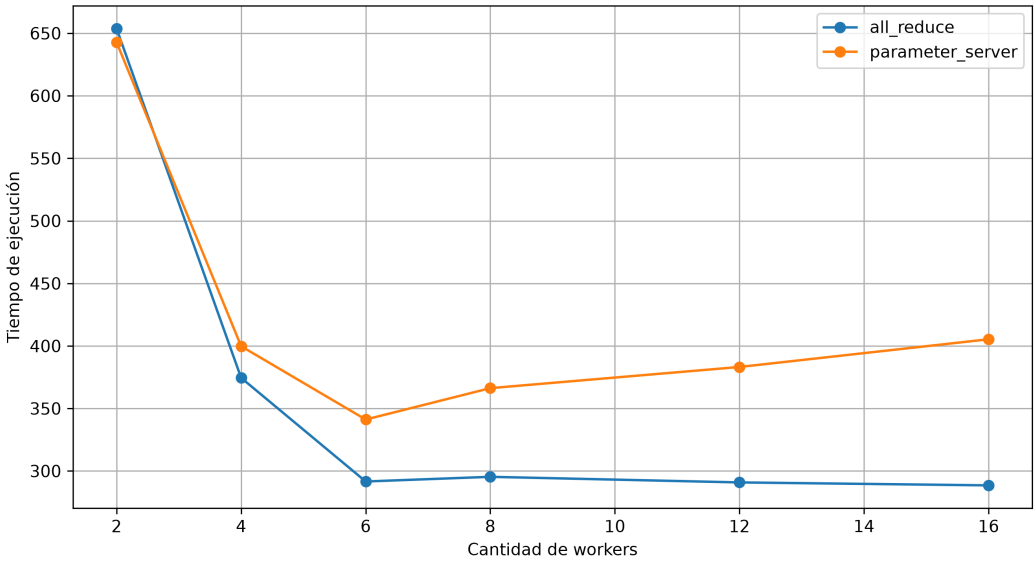


Figura 4.1: Tiempo de entrenamiento de MNIST vs. cantidad de workers para All-Reduce y Parameter Server.

Tal como predice el análisis teórico, la performance de Parameter Server empeora a partir de $N_W = 6$. Esto se debe a que el servidor único actúa como cuello de botella y el delay de sincronización crece linealmente con los workers. En contraste, Ring All-Reduce demuestra ser una topología mucho más eficiente a medida que el sistema escala, puesto que el tiempo de ejecución no se degrada porque su delay está acotado.
Sin embargo, esta sincronización estricta acota la aceleración que proveen estos algoritmos por sobre la ejecución en una sola computadora. Al agregar workers, el término de cómputo T_{epoch}/N_W tiende a cero, pero $d_{\text{sync}}^{\text{AR}}$ tiende a $2S/R$, de modo que el tiempo total nunca baja de $N_{\text{epochs}} \cdot 2(1 - \frac{1}{N_W})\frac{S}{R}$, y a partir de cierta cantidad de workers deja de disminuir.

A continuación se muestran los resultados obtenidos para las estrategias asincrónicas: Parameter Server asincrónico con un solo server y Strategy-Switch.

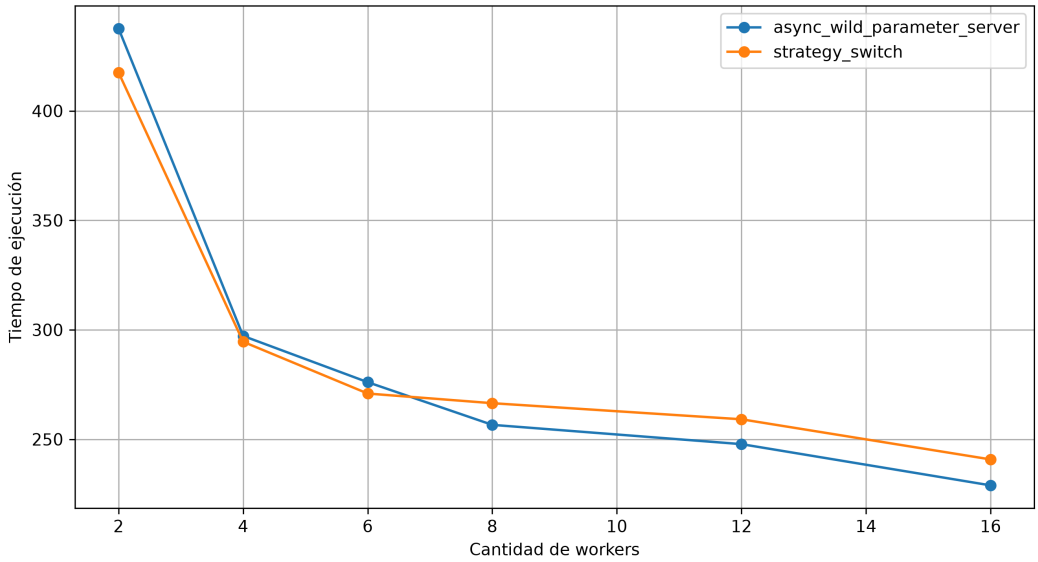


Figura 4.2: Tiempo de entrenamiento de MNIST vs. cantidad de workers para Parameter Server asincrónico y Strategy-Switch.

En términos exclusivos de velocidad, ambos algoritmos logran resultados excelentes. Al romper la barrera de sincronización estricta, evitan el cuello de botella de Ring All-Reduce y logran que el tiempo de ejecución siga bajando a medida que se suman workers.
No obstante, el tiempo de ejecución carece de valor si el modelo no aprende. Al examinar cómo evoluciona la accuracy obtenida sobre el set de test en Parameter Server asincrónico, se observa que la acumulación de gradientes de manera descontrolada arruina por completo los resultados de convergencia, como muestra la figura siguiente.

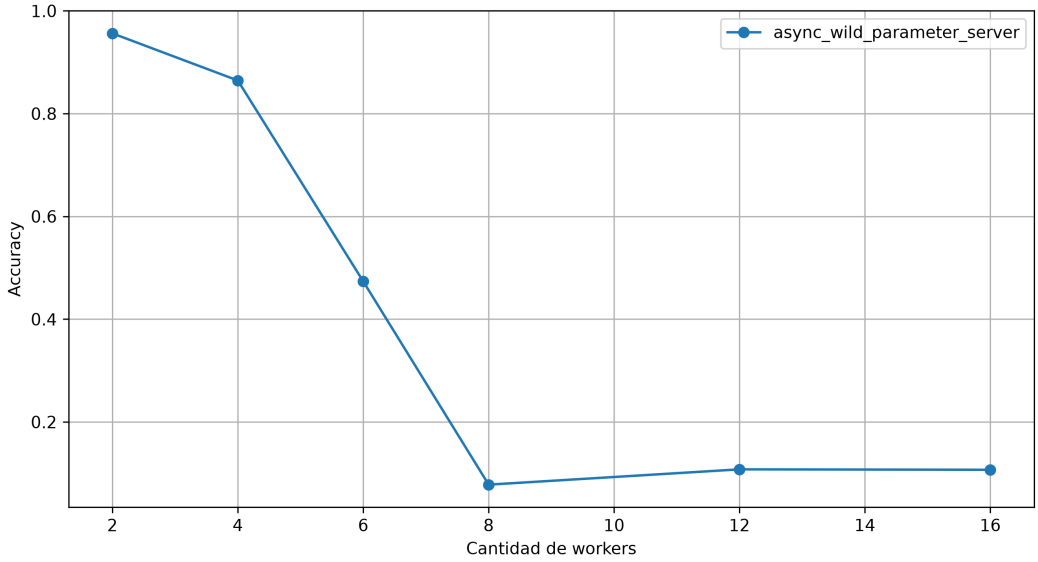


Figura 4.3: Exactitud final sobre el conjunto de prueba de MNIST vs. cantidad de workers para Parameter Server asincrónico.

El hecho de que cada worker empuje para su lado pesa mayormente al principio del entrenamiento, cuando los workers no se encuentran en mínimos locales acordados en conjunto.
Strategy-Switch aprovecha la velocidad de la desincronización en el momento indicado para no perder la convergencia, cuando los mínimos locales sí fueron consensuados de antemano.

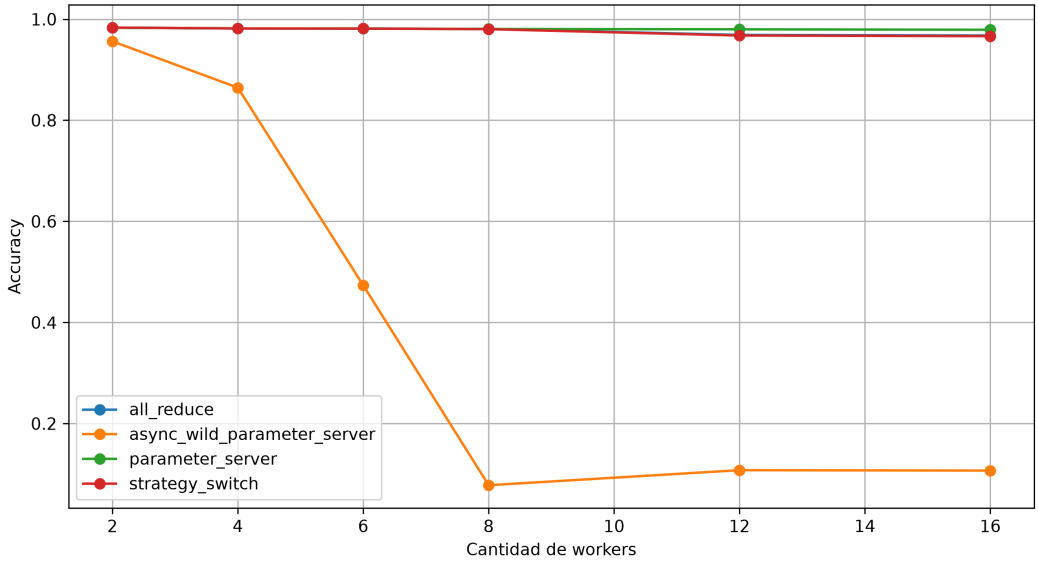


Figura 4.4: Exactitud final sobre el conjunto de prueba de MNIST vs. cantidad de workers para las cuatro estrategias.

En esta campaña, el híbrido obtiene el mejor balance de los evaluados: logra una exactitud similar a la de los algoritmos sincrónicos y una velocidad de ejecución similar a la del asincrónico.

Este análisis tiene una salvedad que conviene explicitar. En esta campaña, Parameter Server asincrónico se ejecutó con `WildStore`, la variante de actualización libre de bloqueos que escribe sobre los parámetros compartidos sin protección y cuyas limitaciones se describen en el Anexo C.2. El colapso de exactitud observado es consistente con el *client drift*, pero estos datos no permiten descartar que las escrituras concurrentes sin protección contribuyan a él. Por lo tanto, los resultados de Parameter Server asincrónico de esta campaña corresponden a esa variante y no al almacenamiento con bloqueo por fragmento (`BlockingStore`).

4.4 Estudio derivado: impacto de los epochs offline

La comparación anterior mantuvo fijos todos los parámetros del entrenamiento distribuido salvo la estrategia. Esta sección hace lo contrario sobre un único parámetro: fija la estrategia y varía los *epochs offline* (E), la cantidad de epochs adicionales que cada nodo entrena de forma autónoma antes de sincronizar: con E epochs offline, cada ronda de sincronización abarca $E + 1$ epochs de entrenamiento local. Es un estudio derivado, en el sentido de que no compara estrategias entre sí sino que aprovecha la instrumentación de O.N.O. para medir el efecto aislado de una decisión de configuración; se desarrolló originalmente como una de las monografías individuales de la materia Aprendizaje Profundo y se reproduce aquí porque valida que la base sirve también para esa clase de preguntas.

4.4.1 Pregunta e hipótesis

Permitir que los nodos operen de forma autónoma durante varios epochs posterga el intercambio de parámetros, disminuyendo la dependencia de la red y priorizando el cómputo local. Sin embargo, demorar la sincronización introduce divergencia entre los pesos de los nodos, un fenómeno conocido como *client drift* (McMahan et al., 2017). O.N.O. expone este mecanismo como un parámetro de configuración, los epochs offline (E), lo que permite estudiarlo de forma directa.

La pregunta es: ¿Cómo impacta la variación de los epochs offline E en la velocidad de convergencia, el tiempo de comunicación y la eficacia del modelo entrenado bajo distintas topologías de red en un entorno distribuido con restricciones de red reales?

Las hipótesis de partida fueron tres. La primera, que incrementar E reduce el tiempo de ejecución, porque reduce en la misma proporción la cantidad de sincronizaciones. La segunda, que incrementar E degrada la exactitud final, por el *client drift* entre réplicas. La tercera, que esa degradación se manifiesta como inestabilidad de la pérdida, y que esa inestabilidad es mayor en All-Reduce que en Parameter Server, porque en el esquema descentralizado no existe un servidor que concentre el estado del optimizador.

4.4.2 Metodología

Se ejecutaron entrenamientos sobre una misma arquitectura, conjunto de datos e hiperparámetros, variando únicamente el algoritmo distribuido, la cantidad de nodos y el valor de E. Todos los ensayos se realizaron de forma secuencial sobre la misma máquina, sumando un total de 11 horas y 15 minutos de ejecución.

Tabla 4.5: Configuración de los ensayos. En Parameter Server, la mitad de los nodos son workers y la otra mitad servidores.

Categoría	Parámetro	Valores
Modelo y datos	Dataset	MNIST completo (LeCun et al., 1998)
	Arquitectura	LeNet-5

Categoría	Parámetro	Valores
	Función de pérdida	Entropía cruzada
Optimización	Optimizador	Adam ($\alpha = 0{,}01$; $\beta_1 = 0{,}9$; $\beta_2 = 0{,}999$; $\epsilon = 10^{-8}$)
	Tamaño de mini-batch	64
	Epochs máximos	48
Infraestructura	Serialización	Dispersa ($r = 0{,}5$)
Variables libres	Algoritmo	All-Reduce, Parameter Server sincrónico
	Nodos	2, 4, 6, 8, 10
	Epochs offline	0, 1, 2, 3, 4

Al parametrizar $E > 0$, los nodos entrenan aislados durante múltiples ciclos y el tráfico de red se reduce de forma aproximada inversamente proporcional a E. Al alcanzar la etapa de sincronización, la consolidación se realiza por promedio directo de los gradientes parciales, de modo que bajo descenso por gradiente la actualización responde a

$$W_{t+1} = W_t - \frac{\lambda}{N} \sum_{i=1}^{N} G_i^{(E)},$$

donde W_t son los parámetros del modelo tras la t-ésima sincronización, λ es la tasa de aprendizaje, N es la cantidad de workers y $G_i^{(E)}$ es el gradiente que el worker i acumuló a lo largo de las $E + 1$ epochs locales de la ronda. Entre dos sincronizaciones consecutivas transcurren entonces $E + 1$ epochs.

Los ensayos se realizaron con la serialización dispersa que implementa el sistema, siguiendo el enfoque de Aji y Heafield (2017). El parámetro r es la fracción de componentes del gradiente que se transmiten: el umbral se fija en el percentil $(1 - r)$ de las magnitudes del gradiente local, de modo que solo viajan, aproximadamente, los $r \cdot |g|$ componentes de mayor magnitud absoluta. Los componentes que no superan el umbral no se descartan: permanecen en un gradiente residual que el worker conserva entre rondas. Los componentes enviados se ponen en cero en ese residual, y el gradiente de la ronda siguiente se suma sobre él antes de recalcular el umbral, de modo que un componente pequeño pero persistente termina superándolo en alguna ronda posterior. Con $r = 0{,}5$, la mitad de los componentes queda fuera de cada mensaje.

El entorno de ejecución, una máquina de cuatro núcleos físicos sobre la que Pumba emuló una red hogareña típica en retardo, *jitter* y ancho de banda, se detalla en la Sección 4.1.2.

4.4.3 Resultados: tiempo de ejecución

La variación de E no indujo mejoras significativas en los tiempos de ejecución: las curvas de cada configuración se solapan de forma estricta en todos los ensayos. Dado que el sistema se ejecuta en una única máquina, la supresión del costo de comunicación no alcanza a compensar la sobrecarga de cómputo, contrariamente a la expectativa inicial.

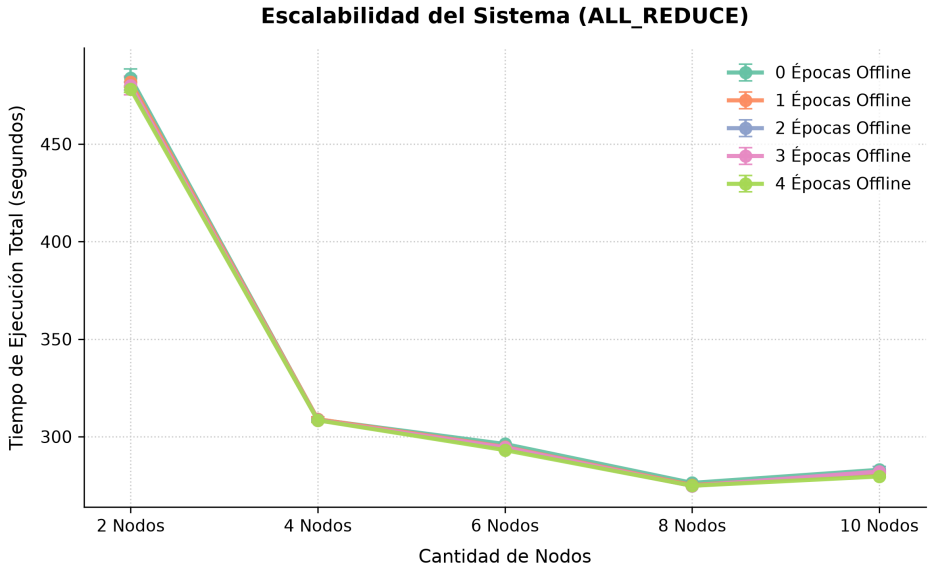


Figura 4.5: Comparación de tiempos de ejecución bajo distintas cantidades de epochs offline con All-Reduce.

La escalabilidad del rendimiento se estanca al superar los 4 nodos, y en All-Reduce la elevación a 10 nodos degrada el desempeño global por la alta contención de CPU.

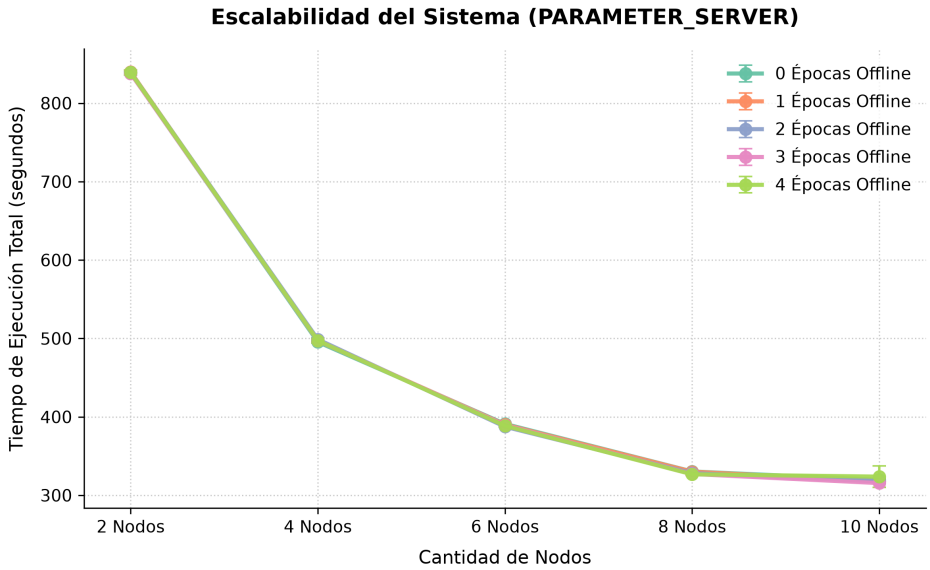


Figura 4.6: Comparación de tiempos de ejecución bajo distintas cantidades de epochs offline con Parameter Server.

En Parameter Server, en cambio, la configuración de 10 nodos (5 workers y 5 servidores) reduce el tiempo respecto de configuraciones menores. La explicación es contraintuitiva pero consistente con el entorno: mientras que en All-Reduce los procesos operan en cómputo continuo, el esquema

sincrónico de Parameter Server introduce estados de inactividad obligatorios mediante barreras, y esa inactividad disminuye la contención, permitiendo una alternancia más eficiente de los hilos de ejecución sobre los cuatro núcleos físicos disponibles. All-Reduce resulta, no obstante, más veloz que Parameter Server en todas las configuraciones evaluadas, por su distribución balanceada de responsabilidades sin dependencias centralizadas; la diferencia es máxima con 2 nodos y se reduce a medida que crece su cantidad.

4.4.4 Resultados: exactitud

El impacto de E se manifiesta con mayor claridad en la exactitud, por ser una métrica independiente de las limitaciones del hardware de ejecución. En ambos algoritmos, la exactitud final decrece de forma monótona al incrementar E. La introducción de un único epoch offline ($E = 1$), que reduce la frecuencia de sincronización global a la mitad, ya provoca un deterioro inmediato de aproximadamente un punto porcentual.

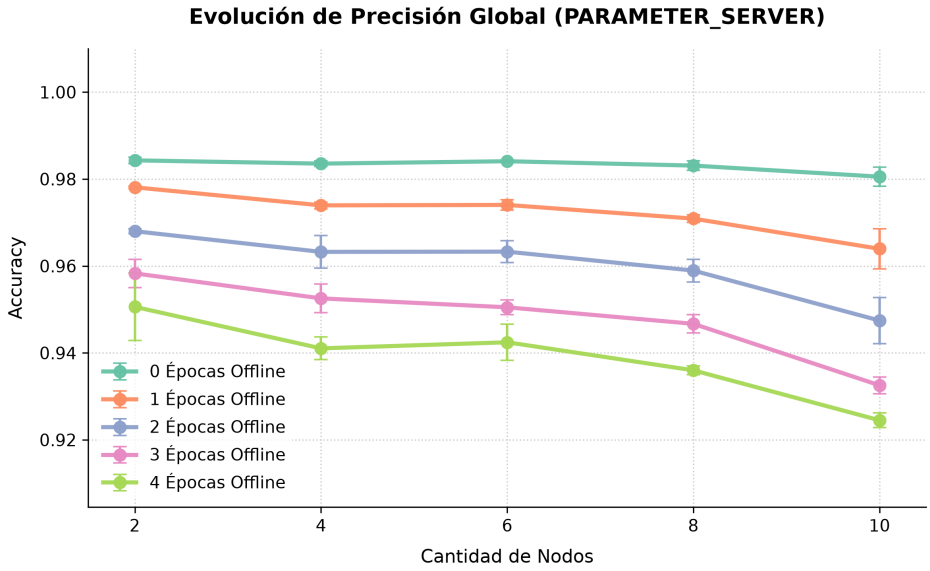


Figura 4.7: Comparación de exactitud bajo distintas cantidades de epochs offline con Parameter Server.

La degradación se acentúa de forma proporcional al número de nodos. Al incrementar las entidades distribuidas, las particiones locales del conjunto de datos se reducen y se vuelven potencialmente sesgadas; la falta de sincronización frecuente exacerba el impacto de esos sesgos locales, perjudicando la convergencia global.

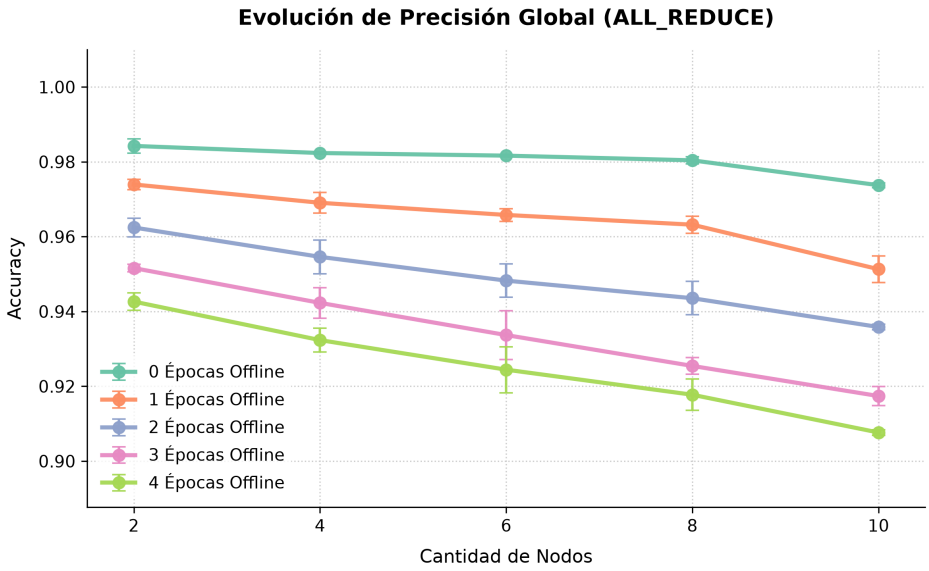


Figura 4.8: Comparación de exactitud bajo distintas cantidades de epochs offline con All-Reduce.

En All-Reduce la pérdida de exactitud es más severa que en Parameter Server. Esto se explica por el diseño experimental: bajo las condiciones evaluadas, All-Reduce duplica el número de workers activos al prescindir de servidores dedicados, lo que fragmenta aún más el conjunto de datos e incrementa el aislamiento operativo. La integración tardía de parámetros fuerza entonces correcciones abruptas del gradiente global.

4.4.5 Resultados: inestabilidad de la pérdida

A partir del historial de entrenamiento se seleccionaron trayectorias representativas que confirman que el incremento de E introduce inestabilidad en la función de pérdida, evidenciada mediante oscilaciones y picos proporcionales al valor de E.

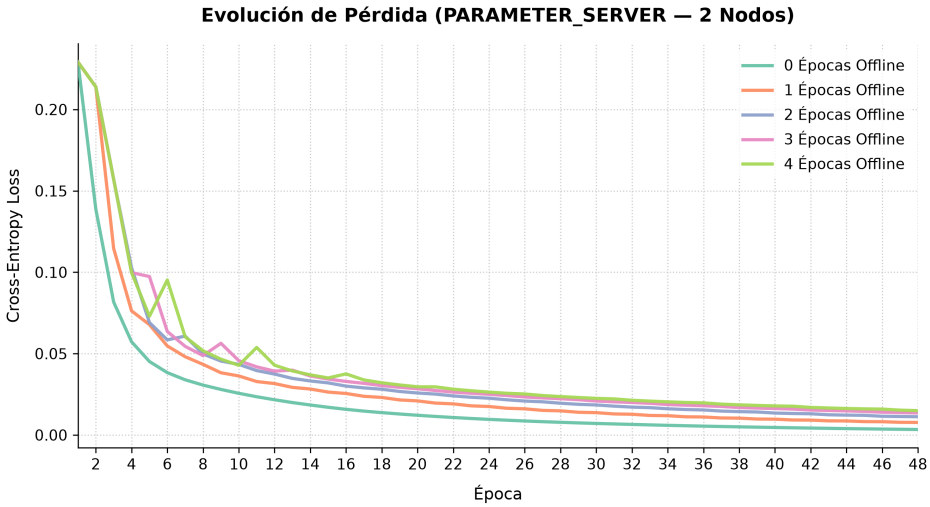


Figura 4.9: Trayectoria de pérdida con entropía cruzada exhibiendo ruido estocástico por cómputo offline en Parameter Server (2 nodos).

La dinámica del optimizador presenta mayores dificultades en los epochs iniciales, la fase asociada a las correcciones de mayor magnitud, y luego se estabiliza al aproximarse a un mínimo local. Sin embargo, los valores de convergencia residual son sistemáticamente más altos a mayor E. El fenómeno sugiere que el desacoplamiento prolongado restringe la capacidad del optimizador para alcanzar el mínimo, induciendo un comportamiento análogo al de una tasa de aprendizaje excesivamente alta.

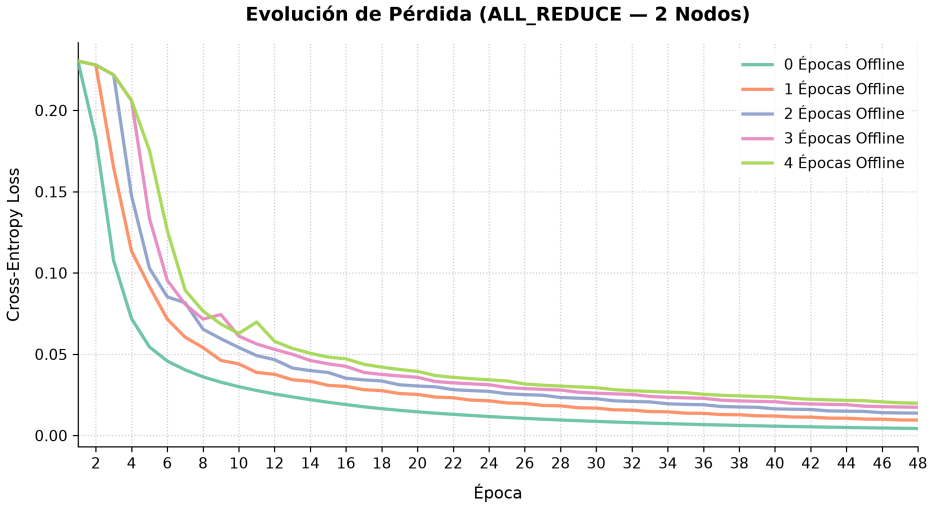


Figura 4.10: Trayectoria de pérdida con entropía cruzada exhibiendo ruido estocástico por cómputo offline en All-Reduce (2 nodos).

En All-Reduce se observa un patrón similar, aunque con oscilaciones relativamente más atenuadas, lo que contradice la tercera hipótesis planteada en la Sección 4.4.1, que preveía mayor inestabilidad en el esquema descentralizado.

4.4.6 Discusión y limitaciones

La ventaja de introducir epochs offline es estrictamente reducir el tiempo de entrenamiento. En un entorno local simulado, reducir el costo de comunicación no compensa el costo de obtener un peor modelo: en estos experimentos no se evidenció ninguna ventaja de utilizar un E mayor a 0.

En contraposición, en un despliegue multimáquina sobre una red física, retrasar la agregación de gradientes puede resultar crítico para amortizar el tiempo de comunicación. La decisión depende de tres factores: el tamaño del modelo, la velocidad de la red y la cantidad de nodos. Con modelos más grandes, los mensajes que contienen gradientes y parámetros son también más grandes. Para entrenamientos con mucho dato y poco modelo, el cuello de botella es el cómputo de gradientes; en cambio, para entrenamientos con poco dato y mucho modelo, donde la comunicación toma protagonismo, incrementar E puede ser útil para reducir los tiempos de ejecución.

Un resultado del estudio que merece señalarse es que la desestabilización afecta negativamente al modelo de forma predecible. Las trayectorias de pérdida muestran, además, que la brecha entre las configuraciones con $E > 0$ y la configuración sin epochs offline es máxima en las primeras epochs, pero no se cierra en las últimas: en las trayectorias presentadas, ninguna configuración con $E > 0$ alcanza la pérdida final de $E = 0$. Esto deja abierta, aunque sin evidencia a favor en estos datos, una línea concreta: evaluar estrategias de activación dinámica de los epochs offline, que los introduzcan solo una vez estabilizada la pérdida, con un criterio análogo al que usa Strategy-Switch para cambiar de régimen, y medir si así se conserva la ganancia en comunicación sin el costo en exactitud que se observó al aplicarlos desde el inicio.

4.4.7 Conclusión del estudio

El uso de epochs offline en O.N.O. altera el balance entre comunicación y fidelidad algorítmica. En la infraestructura simulada, incrementar E perjudicó al modelo entrenado de manera predecible pero sin aportar ganancias de velocidad, debido a las limitaciones del hardware empleado. Queda planteado validar el sistema bajo configuraciones multimáquina reales, comparando arquitecturas de modelos de distintos tamaños al variar E.

4.5 Conclusiones del capítulo

El análisis teórico y empírico de las distintas estrategias de entrenamiento distribuido permite extraer tres conclusiones principales sobre la gestión de tiempo y convergencia:

El cuello de botella es la sincronización centralizada. En escenarios con ancho de banda restringido, centralizar la comunicación en Parameter Server sincrónico escala de forma deficiente y aumenta el tiempo total de entrenamiento al agregar nodos luego de pasar un umbral. La topología descentralizada de Ring All-Reduce soluciona el escalado, pero no elimina el costo de sincronización: su delay por epoch tiende a $2S/R$ al agregar workers, de modo que la aceleración alcanzable sigue acotada.

Existe un *trade-off* directo entre asincronismo y convergencia en Parameter Server asincrónico. Eliminar los bloqueos de sincronización en Parameter Server asincrónico reduce radicalmente los tiempos de ejecución, pero a costa de destruir por completo la convergencia del modelo en este experimento. El comportamiento es consistente con un fuerte *client drift* en las primeras etapas del entrenamiento, con la salvedad, señalada en la Sección 4.3.6, de que esta variante se ejecutó con la actualización libre de bloqueos y no puede descartarse que las escrituras concurrentes contribuyan al colapso.

Superioridad de la estrategia híbrida. Strategy-Switch resultó ser la estrategia más eficiente de las evaluadas sobre MNIST. Al comportarse de forma sincrónica durante la fase crítica del entrenamiento y lograr que los mínimos locales en los que se posicionan los workers se consensuen hasta cierto punto, logra mantener la convergencia del entrenamiento. A su vez, al conmutar al esquema asincrónico luego de ese consenso, rompe el *trade-off* que sí paga Parameter Server asincrónico. La estrategia logra la exactitud de los métodos sincrónicos, equivalentes a la ejecución serial en cuanto a convergencia, en una fracción del tiempo, lo que la ubica, empíricamente, como la mejor alternativa evaluada en esta comparativa.

La observación transversal es que el sistema cumplió su propósito: validada la corrección de las estrategias por la vía de su convergencia, las diferencias de tiempo, throughput y escalabilidad pueden atribuirse al algoritmo y no a su implementación, con la única excepción ya señalada para Parameter Server asincrónico, que era exactamente el problema que este trabajo se propuso resolver.

Capítulo 5

Conclusiones y Trabajo Futuro

Este capítulo cierra el informe con los impactos económico, social y ambiental del proyecto, las líneas de trabajo que quedaron abiertas y las conclusiones generales del trabajo.

5.1 Impactos económico, sociales y ambientales del proyecto

La propuesta incluyó una evaluación preliminar de impacto. Se la revisa aquí a la luz de lo efectivamente construido y medido.

Impacto económico. Es el que este sistema comparte con cualquier plataforma de cómputo distribuido: aprovechar varias máquinas de capacidad modesta que ya se tienen, en lugar de adquirir una sola de gama alta. Esa fue la premisa de partida del trabajo, pero el trabajo no la demuestra: afirmar que la alternativa distribuida resulta más económica exigiría comparar el costo real del equipamiento que produce resultados similares en cada caso, comparación que no se realizó y queda como línea futura. La ganancia depende de que la distribución de la carga de cómputo por nodo justifique el costo de sumarlos. El aporte económico del sistema es, entonces, permitir ubicar ese umbral antes de invertir en infraestructura. El proyecto en sí se desarrolló sobre hardware propio y con herramientas de código abierto, de modo que su costo se reduce a las 1590 horas-persona que se detallan en *Aportes individuales*.

Impacto social. El sistema se publica como base abierta y parametrizable, con implementaciones de referencia de los tres algoritmos sobre una misma infraestructura, una biblioteca de Python y una interfaz de terminal que muestra el entrenamiento en vivo. El objetivo es acercar el entrenamiento distribuido a equipos sin grandes clústeres y ofrecer un punto de partida para la investigación y la docencia. Su valor pedagógico resultó más concreto de lo previsto: al no delegar nada en un framework existente, expone la mecánica completa del entrenamiento distribuido, desde la propagación hacia atrás hasta el protocolo de red.

Impacto ambiental. El impacto ambiental del sistema es su consumo energético, que no se midió. Frente a un único equipo de la misma capacidad que cada nodo, un entrenamiento sobre N nodos consume más energía agregada aun cuando termine antes, porque suma la comunicación y la coordinación entre nodos. Frente a un equipo más veloz que realice el mismo entrenamiento de forma secuencial en el mismo tiempo que el sistema distribuido, en cambio, el signo de la diferencia

depende de la eficiencia energética de cada equipo y no puede afirmarse sin medirla. La distribución se justifica ambientalmente cuando permite entrenar modelos que de otro modo no serían viables o cuando se despliega sobre máquinas que de todas formas estarían encendidas, y las técnicas de reducción de tráfico implementadas apuntan a acortar la comunicación, que es parte de ese consumo adicional. Medir ambos balances, el económico y el energético, queda como línea futura.

5.2 Trabajos futuros

El sistema quedó en un estado que admite ser retomado, y varias de las líneas que siguen tienen su diseño ya especificado en el repositorio. Se ordenan por relevancia.

Tolerancia a fallos. El trabajo planteó la ejecución del entrenamiento distribuido sobre un entorno local, donde la probabilidad de fallos se consideró muy baja, y por eso la funcionalidad no se abordó. Resulta, sin embargo, imprescindible para escalar un sistema de estas características a un entorno no local. Y aún dentro del entorno contemplado por el trabajo, donde los fallos de hardware son poco probables, quedan desatendidos los errores de usuario y los imprevistos cotidianos: una máquina que se queda sin batería, alguien que patea un cable de alimentación o que interrumpe un proceso sin querer.
El comportamiento esperado ya está especificado por entidad: el orchestrator debe encolar los mensajes destinados a un par caído hasta que este se reconecte; el servidor de parámetros debe distinguir un worker lento de uno muerto; y el worker de All-Reduce debe poder esperar la reconexión de sus vecinos en el anillo. La pieza habilitante es una capa de reconexión en el transporte.
Un requisito adicional ya relevado: dado que el sistema admite varias entidades sobre la misma dirección IP con puertos distintos, la reconexión necesita un servicio de resolución de nombres por el cual cada extremo anuncie, al establecerse la conexión, el puerto en el que escuchará si se cae y revive.

Resolver la actualización de parámetros libre de bloqueos de *HogWild!*, o retirarla. La variante actual no logra reunir tres propiedades a la vez: admitir cualquier optimizador, incluidos los que tienen estado; ser realmente libre de bloqueos; y producir actualizaciones correctas. Cualquier solución debe resignar al menos una de ellas, y hay dos caminos posibles. Las operaciones atómicas dan corrección, pero solo con un optimizador sin estado, porque el ciclo de lectura-modificación-escritura de Adam o de momentum no admite ser libre de bloqueos; un bloqueo por fragmento lo resuelve, pero deja de ser la técnica del paper. Corresponde elegir de forma explícita en lugar de dejar la variante en el limbo, y cualquiera de las dos exige antes verificar empíricamente si el enfoque rinde con gradientes densos, es decir, gradientes en los que la mayoría de los componentes son distintos de cero, que es el régimen en el que opera el sistema con redes convolucionales y densas. La garantía de convergencia de *Hogwild!* se apoya en la hipótesis opuesta, la de actualizaciones dispersas que modifican pocos parámetros a la vez, y por eso no puede trasladarse sin más a este régimen. Esta verificación incluye repetir la campaña de Parameter Server asincrónico de la Sección 4.3.6 con almacenamiento con bloqueo, para separar el efecto del *client drift* del de las escrituras concurrentes.

Variación dinámica del modelo de cómputo de la convolución. El motor ya reparte entre hilos las operaciones que se aplican capa por capa, como la actualización de parámetros y la normalización del gradiente, pero el cálculo de la convolución en sí es secuencial. Paralelizarlo llegó a implementarse y se descartó porque agregaba más sobrecarga que beneficio, con una conclusión acotada a MNIST y a modelos pequeños. También se probó utilizar convoluciones por transformadas rápidas de Fourier (Mathieu, Henaff, & LeCun, 2013) pero el *overhead* introducido por dicha estrategia también resultó demasiado para la ganancia. Con modelos, batches y kernels considerablemente mayores el balance podría invertirse, y el trabajo dejó la medición hecha como línea de base contra la cual comparar. Se deja como trabajo futuro variar las estrategias según la dimensionalidad de los datos que llegan a las capas convolucionales que los procesan y la dimensionalidad de sus kernels.

Consumo de conjuntos de datos por flujo. El sistema carga las particiones del conjunto de datos en memoria. Un mecanismo de lectura por flujo desde disco permitiría trabajar con conjuntos que no entran en la memoria de un nodo, que es uno de los dos motivos originales por los que se distribuye un entrenamiento y el que este trabajo no llegó a ejercitar.

Ampliación del motor de redes neuronales. La biblioteca cuenta con el conjunto mínimo y necesario para ejercitar los modelos con los que se propuso resolver MNIST y validar el sistema. Queda pendiente ampliarla con más funciones de activación y extenderla a otras arquitecturas, como las redes recurrentes, las de tipo generativo y las que no forman una cadena de capas, como U-Net (Ronneberger et al., 2015), una red convolucional para segmentación de imágenes cuyas conexiones de salto entre capas no contiguas exigen que el motor admita grafos de capas y no solo secuencias.

Líneas acotadas con su alcance ya definido. En una sesión de definición de alcance previa al cierre se dejaron fuera, de forma explícita, tres extensiones que el sistema admite sin cambios estructurales: el multiplexado de sesiones sobre una misma conexión, el consumo de conjuntos de datos por flujo y la parametrización genérica del tipo de punto flotante. Cada una es autocontenida y puede retomarse por separado.

Evaluación aislada de las técnicas de reducción de tráfico. El sistema implementa codificación de gradientes en media precisión y un protocolo de gradientes dispersos con acumulación del residuo. Ambas se utilizaron en los experimentos, pero no se evaluaron de forma aislada: no se midió el compromiso entre la tasa de compresión y la degradación de la convergencia. Es un estudio autocontenido, de alcance acotado y con toda la instrumentación ya disponible.

5.3 Conclusiones

El trabajo se propuso estudiar las principales estrategias de entrenamiento distribuido de modelos de aprendizaje profundo y construir un sistema que permitiera compararlas de forma controlada. Ese objetivo se cumplió.

Sobre el producto. Se construyó O.N.O., un sistema distribuido de entrenamiento que incluye una biblioteca de redes neuronales propia, una capa de comunicación con su propio protocolo, un plano de control y dos interfaces de uso. Sobre esa base se implementaron los tres algoritmos de referencia del área, se agregaron técnicas de reducción del tráfico de red y se instrumentó todo con una suite de experimentación reproducible.

Sobre los resultados. Los estudios experimentales realizados sobre MNIST muestran que los algoritmos sincrónicos convergen con buena exactitud a costa de un mayor tiempo de entrenamiento, que *Parameter Server asincrónico* es más rápido pero a costa de exactitud y que *Strategy-Switch* logra combinar las ventajas de ambos mundos exitosamente; y que postergar la sincronización entre nodos mediante epochs offline degrada la exactitud final de forma monótona sin aportar, en ese entorno, la ganancia de tiempo que la motiva. La elección de la estrategia en un despliegue real depende también de la heterogeneidad del hardware de los nodos y de la relación entre el tamaño del modelo y el ancho de banda de la red. Estas conclusiones valen para MNIST, con la arquitectura LeNet-5 y el entorno simulado descritos en el capítulo de experimentación. Para afirmar qué estrategia conviene en general, hacen falta más mediciones con otros problemas y conjuntos de datos, modelos de otros tamaños y una red física real.

Sobre la contribución. El trabajo no innova en algoritmos: los tres implementados están publicados. La contribución es el instrumento, y responde exactamente al problema detectado en la propuesta: la ausencia de una base común sobre la cual comparar estrategias sin que la diferencia medida mezcle el efecto del algoritmo con el de su implementación. La evidencia de que esa base funciona es el conjunto de benchmarks obtenidos.

Sobre la formación. El trabajo obligó a atravesar de punta a punta tres dominios que la carrera aborda por separado. El de los sistemas distribuidos, el del aprendizaje profundo, que dejó una comprensión del entrenamiento que ningún framework habría permitido adquirir, y el de la ingeniería de software, donde la lección central fue que la instrumentación no es un anexo de medición sino una herramienta de detección, según se detalla en *Riesgos materializados y lecciones aprendidas* (Anexo C.2).

Se cierra el trabajo con un sistema funcionando, con evidencia experimental que lo respalda y con las líneas que quedaron abiertas documentadas al detalle suficiente para que otro las retome.

Referencias

Aji, A. F., & Heafield, K. (2017). Sparse Communication for Distributed Gradient Descent. *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 440-445. https://doi.org/10.18653/v1/D17-1045

Ben-Nun, T., & Hoefler, T. (2019). Demystifying Parallel and Distributed Deep Learning: An In-Depth Concurrency Analysis. *ACM Computing Surveys*, *52*(4). https://doi.org/10.1145/3320060

Dehghani, M., & Yazdanparast, Z. (2023). From Distributed Machine to Distributed Deep Learning: A Comprehensive Survey. *arXiv preprint arXiv:2307.05232*. Recuperado de https://arxiv.org/abs/2307.05232

Glorot, X., & Bengio, Y. (2010). Understanding the Difficulty of Training Deep Feedforward Neural Networks. *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics*, 249-256.

He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. *Proceedings of the IEEE International Conference on Computer Vision*, 1026-1034.

Held, M., & Karp, R. M. (1962). A Dynamic Programming Approach to Sequencing Problems. *Journal of the Society for Industrial and Applied Mathematics*, *10*(1), 196-210. https://doi.org/10.1137/0110015

Jiang, Y., Zhu, Y., Lan, C., Yi, B., Cui, Y., & Guo, C. (2020). A Unified Architecture for Accelerating Distributed DNN Training in Heterogeneous GPU/CPU Clusters. *Proceedings of OSDI '20*.

Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. *arXiv preprint arXiv:1412.6980*. Recuperado de https://arxiv.org/abs/1412.6980

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, *86*(11), 2278-2324.

Li, M., Andersen, D. G., Park, J. W., Smola, A. J., Ahmed, A., Josifovski, V., ... Su, B.-Y. (2014). Scaling distributed machine learning with the parameter server. *Proceedings of the 11th USENIX Conference on Operating Systems Design and Implementation*, 583-598. USA: USENIX Association.

Li, Z., Davis, J., & Jarvis, S. (2017). An Efficient Task-based All-Reduce for Machine Learning Applications. *Proceedings of the Machine Learning on HPC Environments*. New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3146347.3146350

Lin, Y., Han, S., Mao, H., Wang, Y., & Dally, W. J. (2018). Deep Gradient Compression: Reducing the Communication Bandwidth for Distributed Training. *International Conference on Learning Representations*. Recuperado de https://arxiv.org/abs/1712.01887

Mathieu, M., Henaff, M., & LeCun, Y. (2013). *Fast Training of Convolutional Networks through FFTs*. https://doi.org/10.48550/arXiv.1312.5851

McMahan, B., Moore, E., Ramage, D., Hampson, S., & Arcas, B. A. y. (2017). Communication-

Efficient Learning of Deep Networks from Decentralized Data. *Artificial Intelligence and Statistics*, 1273-1282. PMLR.

Niu, F., Recht, B., Ré, C., & Wright, S. J. (2011). Hogwild!: A Lock-Free Approach to Parallelizing Stochastic Gradient Descent. *Advances in Neural Information Processing Systems 24*. Recuperado de https://arxiv.org/abs/1106.5730

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... Chintala, S. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Recuperado de https://arxiv.org/abs/1912.01703

Patarasuk, P., & Yuan, X. (2009). Bandwidth Optimal All-reduce Algorithms for Clusters of Workstations. *Journal of Parallel and Distributed Computing*, *69*(2), 117-124. https://doi.org/10.1016/j.jpdc.2008.09.002

Prechelt, L. (1998). Early Stopping — But When? En *Neural Networks: Tricks of the Trade* (pp. 55-69). Springer. https://doi.org/10.1007/3-540-49430-8_3

Provatas, N., Chalas, I., Konstantinou, I., & Koziris, N. (2025). Strategy-Switch: From All-Reduce to Parameter Server for Faster Efficient Training. *IEEE Access*, *PP*, 1-1. https://doi.org/10.1109/ACCESS.2025.3528248

Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, 234-241. Springer. https://doi.org/10.1007/978-3-319-24574-4_28

Sergeev, A., & Del Balso, M. (2018). Horovod: Fast and Easy Distributed Deep Learning in TensorFlow. *arXiv preprint arXiv:1802.05799*. Recuperado de https://arxiv.org/abs/1802.05799

Sutskever, I., Martens, J., Dahl, G., & Hinton, G. (2013). On the Importance of Initialization and Momentum in Deep Learning. *Proceedings of the 30th International Conference on Machine Learning*, 1139-1147.

TensorFlow Developers. (2021). *TensorFlow*. Zenodo. https://doi.org/10.5281/zenodo.4758419

Anexo A

Aportes individuales

El desarrollo del trabajo se planteó como una responsabilidad compartida: las etapas de análisis, revisión de código y pruebas fueron llevadas adelante por los tres integrantes en todas las tareas. Sin perjuicio de ello, cada integrante concentró su foco principal en un conjunto de componentes, que se detalla a continuación y que se corresponde con lo previsto en la propuesta.

Alejo Ordoñez (Padrón 108397). Núcleo de aprendizaje automático: el modelo, las capas, los optimizadores y las funciones de pérdida. Las capas convolucionales y de *max-pooling*, incluida la reformulación de la convolución como multiplicación de matrices que produjo la mejora final de rendimiento del motor. El manejo y la distribución de los conjuntos de datos.

Lorenzo Minervino (Padrón 107863). El módulo del Worker, con participación en la implementación en anillo de *All-Reduce*. La interfaz de funciones externas en Python y la interfaz interactiva de terminal. Las optimizaciones de sincronización de las copias del modelo. Los entornos de simulación multinodo, la suite de benchmarks y el análisis experimental de resultados.

Marcos Bianchi Fernández (Padrón 108921). Los algoritmos de entrenamiento distribuido y su coordinación. El módulo de comunicaciones y el protocolo entre nodos, incluidas las técnicas de reducción de tráfico. El módulo del Servidor de parámetros. La monografía sobre el impacto de los epochs offline, que constituye el estudio experimental derivado del informe.

Tabla A.1: Horas dedicadas por cada integrante a cada tarea del plan de actividades.

	Alejo	Lorenzo	Marcos
Cantidad de horas insumidas	520	545	525
Leer y analizar los trabajos previos	25	20	15
Investigar la implementación distribuida de TensorFlow	0	5	0
Investigar la implementación distribuida de PyTorch	20	0	5
Desarrollar el sistema distribuido en Rust	120	70	210
Implementar Parameter Server	90	10	100
Implementar All-Reduce	10	20	20
Implementar Strategy-Switch	15	0	35

	Alejo	Lorenzo	Marcos
Optimizaciones de comunicación entre nodos	50	0	50
Optimizaciones de sincronización de las copias del modelo	20	80	0
Interfaz de funciones externas en Python e interfaz de terminal	0	100	0
Pruebas unitarias y de integración	20	60	20
Simulación sobre distintas configuraciones de nodos	30	100	20
Optimizaciones de carga en configuraciones heterogéneas (no ejecutada)	0	0	0
Análisis de resultados y gráficos	50	40	10
Documentación del código y del proceso	60	10	30
Informe final	10	30	10

Anexo B

Metodología de trabajo aplicada

Este anexo actualiza lo planteado en la propuesta, contrastando lo previsto con lo efectivamente ejecutado en la gestión del proyecto.

B.1 Gestión y roles

Se estableció un compromiso de 500 horas por estudiante a lo largo de dos cuatrimestres, lo que representa un promedio de unas 16 horas semanales por persona sobre 32 semanas. El Ing. Ricardo A. Veiga cumplió la función de tutor y el Dr. Ing. J. Ignacio Alvarez-Hamelin la de co-tutor y orientador en el campo de desempeño, en el marco del CoNexDat (Grupo de Redes Complejas y Comunicación de Datos de la Facultad de Ingeniería).

Los tres estudiantes participaron de todas las etapas de análisis, implementación y pruebas, y cada integrante concentró su foco principal en un conjunto de componentes, según se detalla en *Aportes individuales*. Esa especialización, que emergió del desarrollo y que no estaba impuesta por la propuesta, tuvo un efecto secundario relevante: la revisión cruzada de código se volvió el principal mecanismo de difusión de conocimiento entre componentes, y en varios casos las discusiones de revisión terminaron modificando decisiones de diseño y no solo detalles de implementación.

La coordinación se sostuvo en dos planos. Con los tutores se mantuvieron reuniones virtuales para informar el avance, definir prioridades y planificar próximas etapas, adecuando los encuentros según los hitos y necesidades del proyecto dentro del marco acordado. Entre los integrantes, la coordinación fue continua y de cadencia diaria, apoyada en un canal de mensajería para las decisiones rápidas y en las issues del repositorio para las que requerían registro.

B.2 Proceso de desarrollo

El método de desarrollo fue incremental, organizado en iteraciones con entregables intermedios. La unidad de entrega fue en forma de *pull requests*: cada funcionalidad se desarrolló en una rama propia que se integraba a la rama principal con previa revisión de los demás integrantes. A lo largo del proyecto se produjeron +1100 commits, +120 pull requests y +85 issues, entre septiembre de 2025 y julio de 2026.

El código y todos los artefactos versionables se gestionaron con Git sobre un repositorio alojado en GitHub, y los mensajes de los commits siguen la convención de *Conventional Commits*. Los entregables intermedios que marcaron el avance fueron, en orden: la capa de comunicación funcionando de punta a punta, el primer entrenamiento distribuido completo con Parameter Server emulando una compuerta lógica XOR, el motor de redes neuronales convergiendo sobre MNIST, la interfaz de terminal mostrando un entrenamiento en vivo, All-Reduce en anillo, la librería FFI de Python, Strategy-Switch, y finalmente la suite de benchmarks con sus resultados.

B.3 Seguimiento, tickets y gestión del alcance

El seguimiento del proceso, la gestión de tickets y el registro de errores se realizaron con las *issues* del repositorio, categorizadas mediante etiquetas por tipo (*feature*, *bug*, *enhancement*, *documentation*) y por componente del sistema (*parameter-server*, *comms*, *worker*, *ffi*, *tui*, entre otras). Las issues se usaron además, y esto excedió lo previsto en la propuesta, como espacio de debate de diseño: varias de las decisiones de arquitectura documentadas en este informe se resolvieron enteramente en el hilo de comentarios de un issue, con enumeración de alternativas y su descarte razonado. Ese registro fue el insumo principal para reconstruir el proceso.

Los errores se clasificaron según su criticidad, separando los que frenaban el trabajo de los que podían corregirse después. La gestión del alcance se definió en dos momentos clave: al inicio, cuando se concibió el MVP, y antes del cierre, en una sesión donde se descartó deliberadamente lo que no se llegaba a desarrollar. Ambos puntos se detallan en la sección de riesgos.

B.4 Automatización

La construcción y las pruebas del sistema se ejecutan con las herramientas del ecosistema de Rust (`cargo test`), y los entornos de simulación se levantan de forma automatizada mediante Docker y un `Makefile` autodocumentado, lo que hace reproducible el despliegue de las distintas configuraciones de nodos.

La construcción, las pruebas, el análisis estático, la ejecución de entornos multinodo y de benchmarks completos quedaron totalmente automatizados y de ejecución desatendida: el conjunto final de 38 configuraciones se ejecutó de un solo comando durante ~6 horas sin intervención. La suite de tests se ejecuta cada vez que se commitea en un Pull Request que mergea en la rama principal mediante GitHub Actions; es obligatorio que la totalidad de las pruebas pase de forma satisfactoria para habilitar el merge.

B.5 Criterios de aceptación

Cada pull request se sometió a revisión de código por parte de otro integrante antes de integrarse. Una entrega se consideró aceptada cuando se cumplían de forma conjunta tres condiciones: la revisión de código fue aprobada, la totalidad de las pruebas automatizadas asociadas se ejecutaba con éxito, y la funcionalidad requerida quedaba demostrada mediante una prueba de aceptación. Para las funcionalidades que involucran coordinación entre nodos, esa prueba consistió en una ejecución

completa de entrenamiento distribuido sobre el entorno simulado, verificando que la convergencia obtenida fuera consistente con la del entrenamiento secuencial equivalente.

El criterio funcionó para las fallas visibles, pero resultó insuficiente para detectar errores de corrección distribuida silenciosos, aquellos que no provocan fallos sino que degradan la calidad del modelo. Este tipo de errores se solventó con la introducción de benchmarks y se detalla en las lecciones aprendidas.

B.6 Artefactos e Indicadores

Como herramienta de gestión se utilizó GitHub Projects. Cada ticket nació en el `Backlog`, pasó `En Progreso` y eventualmente, cuando el feature alcanzó un estado estable pasó a `En Revisión`, para finalmente quedar archivado en `Listo` luego de su efectiva revisión.

Algunos de los artefactos de gestión que se utilizaron durante el proceso fueron minutas de reunión para una mejor organización previa a los encuentros con los tutores y diagramas de los temas que se abarcaron, si lo ameritaban; como por ejemplo diagramas de robustez para los distintos algoritmos distribuidos que se implementaron, y diagramas de ejecución del flujo de entrenamiento agnóstico del tipo de entrenamiento mostrando la comunicación entre orchestrator y workers.

Sobre la calidad del producto se definieron los siguientes indicadores: la proporción de cambios requeridos luego de publicar un **pull request** como listo para revisión, y el desvío entre horas planificadas y efectivamente dedicadas. Este último funcionó también como indicador de costos.

También se consideraron la proporción de pruebas exitosas sobre el total, la cantidad de advertencias del análisis estático y los resultados obtenidos de la ejecución de los benchmarks. En cuanto al análisis estático, se buscó sostenerlo sin warnings durante toda la evolución del proyecto.

Anexo C

Cronograma y riesgos del proyecto

C.1 Cronograma de las actividades realizadas

El plan de actividades de la propuesta fijó dieciséis tareas repartidas en cuatro hitos sobre dos cuatrimestres, con una carga estimada de 1500 horas de equipo. Esta sección contrasta ese plan con la ejecución real, que se extendió entre agosto de 2025 y julio de 2026.

C.1.1 Fases efectivamente recorridas

El desarrollo puede reconstruirse en doce fases, numeradas de 0 a 11. Las fechas son aproximadas y corresponden a rangos de actividad, ya que varias fases se solaparon.

Tabla C.1: Fases del desarrollo efectivamente recorridas.

Fase	Período	Contenido
0	Ago–Nov 2025	Propuesta, relevamiento bibliográfico y un primer diseño descartado
1	Nov 2025–Ene 2026	Capa de comunicación
2	Dic 2025–Feb 2026	Parameter Server y Worker
3	Ene–Mar 2026	Motor de redes neuronales
4	Feb–Mar 2026	Orchestrator, interfaz de terminal e interfaz de Python
5	Mar 2026	Multi-servidor, conjuntos de datos y eficiencia de red
6	Mar–May 2026	All-Reduce en anillo y unificación del nodo
7	Abr–May 2026	Early stopping y la saga de bloqueos
8	May–Jun 2026	Strategy Switch, topología y benchmarks
9	Jun 2026	Endurecimiento y exposición de funcionalidad
10	Jun–Jul 2026	Consolidación del benchmark y auditoría de corrección
11	Jul 2026	Optimización final y documentación

C.1.2 Contraste entre lo planificado y lo ejecutado

Lo que se cumplió según lo previsto. El núcleo del plan se completó: el sistema base parametrizable en Rust con su biblioteca de redes neuronales propia (tarea 4), los tres algoritmos (tareas 5 a 7), las optimizaciones de comunicación y sincronización (tareas 8 y 9), las dos interfaces (tarea 10), las pruebas (tarea 11), la simulación sobre distintas configuraciones de nodos (tarea 12) y el análisis de resultados con sus gráficos (tarea 14). El desvío global fue de noventa horas sobre las mil quinientas estimadas, apenas un seis por ciento, pero ese saldo casi neutro esconde compensaciones grandes entre tareas: 550 horas de exceso en unas se cancelaron contra 460 de defecto en otras.

Lo que llevó más tiempo del estimado. El desvío dominante es la tarea 4, que pasó de 150 a 400 horas y explica por sí sola casi la mitad del exceso. Concentra la construcción del sistema base y, sobre todo, la del motor de redes neuronales: la derivación manual del algoritmo de *backpropagation* y la implementación de las capas convolucionales insumieron alrededor de dos meses de depuración de gradientes, muy por encima de lo previsto para un componente que la propuesta trataba como parte de una tarea mayor. El segundo desvío es el Parameter Server (tarea 5), que duplicó su estimación porque llegó a su diseño definitivo después de tres intentos cerrados. El resto del exceso se reparte de forma pareja entre cuatro tareas de acompañamiento que el plan subestimó por igual, las interfaces (tarea 10), las pruebas (tarea 11), la simulación (tarea 12) y la documentación (tarea 15), cada una del doble o del doble y medio de lo previsto.

Lo que llevó menos. All-Reduce y Strategy-Switch (tareas 6 y 7) costaron la mitad de lo estimado, y la razón es que llegaron sobre infraestructura ya construida: el worker, la capa de comunicación y el plano de control existían desde la segunda fase, de modo que lo que quedaba era el algoritmo y no el andamiaje. Las optimizaciones de comunicación y de sincronización (tareas 8 y 9) también quedaron por debajo, aun habiendo absorbido la saga de bloqueos de coordinación de la fase 7, que no figuraba en el plan bajo ninguna tarea. Conviene señalar además que la fase 0 se extendió por cuatro meses de calendario, fue un período dedicado a documentación, relevamiento y trámites; con un primer diseño de arquitectura descartado por completo.

Lo que no se ejecutó. La tarea 13, optimización de carga de cómputo en configuraciones heterogéneas, con 150 horas estimadas, no se abordó.

Lo que se ejecutó sin estar planificado. Aparecieron cuatro líneas no previstas: la selección de topología mediante estadísticas de latencia, la auditoría de corrección distribuida que surgió de la construcción de la suite de benchmarks, la optimización del motor de convolución, y las dos monografías que constituyen los estudios experimentales de este informe, que excedieron en profundidad lo que la tarea 14 contemplaba.

Una observación sobre la estimación. La distribución de horas de la propuesta asignaba 200 horas a investigar las implementaciones distribuidas de TensorFlow y PyTorch (tareas 2 y 3). En la práctica ese estudio se realizó de forma mucho más acotada y en modalidad de consulta puntual, y el esfuerzo se redistribuyó hacia la implementación y la depuración del motor de aprendizaje profundo. La lección de estimación es la habitual y no por conocida menos real: el esfuerzo se subestima sistemáticamente en aquello que no se conoce de antemano, que en este trabajo fue todo lo relativo a la corrección numérica del entrenamiento y a la coordinación distribuida.

C.1.3 Carga horaria efectiva

Tabla C.2: Contraste entre la carga horaria estimada en la propuesta y la efectivamente dedicada.

Nro.	Tarea	Horas estimadas	Horas reales
1	Leer y analizar los trabajos previos	100	60
2	Investigar la implementación distribuida de TensorFlow	50	5
3	Investigar la implementación distribuida de PyTorch	50	25
4	Desarrollar el motor de aprendizaje profundo	150	400
5	Implementar Parameter Server	100	200
6	Implementar All-Reduce	100	50
7	Implementar Strategy-Switch	100	50
8	Optimizaciones de comunicación entre nodos	150	100
9	Optimizaciones de sincronización de las copias del modelo	150	100
10	Interfaz de funciones externas en Python e interfaz de terminal	50	100
11	Pruebas unitarias y de integración	50	100
12	Simulación sobre distintas configuraciones de nodos	100	150
13	Optimizaciones de carga en configuraciones heterogéneas	150	0 (no ejecutada)
14	Análisis de resultados y gráficos	100	100
15	Documentación del código y del proceso	50	100
16	Informe final	50	50
	Total	1500	1590

La distribución de esas horas entre los tres integrantes se detalla en *Aportes individuales*.

C.2 Riesgos materializados y lecciones aprendidas

La propuesta identificó cuatro riesgos iniciales. Esta sección los enuncia, recorre con la evidencia del historial del proyecto los que se materializaron, suma los que no estaban contemplados y cierra con las lecciones que dejaron.

C.2.1 Riesgos previstos en la propuesta

- **Complejidad de la implementación distribuida en Rust.** La coordinación entre nodos y el manejo de la concurrencia son intrínsecamente complejos y propensos a errores sutiles, como bloqueos o condiciones de carrera. Como mitigación, el desarrollo fue incremental y se apoyó en las garantías de *fearless concurrency* del lenguaje y en una cobertura de pruebas unitarias y de integración que acompañó cada avance.
- **Disponibilidad de hardware para simulaciones representativas.** Reproducir un entorno distribuido realista requiere de múltiples máquinas, algo no siempre disponible. Para mitigarlo,

las simulaciones se ejecutaron sobre contenedores Docker con límites de recursos por nodo, lo que permitió emular entornos multi computadora de forma controlada.

- **Dependencia de trabajos previos.** Parte del trabajo se apoya en algoritmos y resultados publicados, como *Strategy-Switch*, cuya interpretación e implementación pueden diferir de lo documentado. La mitigación consistió en implementar versiones de referencia de los algoritmos base y validarlas antes de construir mejoras sobre ellas.
- **Amplitud del alcance y estimación de tiempos.** El trabajo abarcó tanto el sistema base como varias líneas de optimización (comunicación, sincronización y carga en configuraciones heterogéneas), lo que dificultó la estimación del esfuerzo. Para mitigarlo, se priorizó un sistema base funcional junto con las implementaciones de referencia, y las optimizaciones se abordaron de forma incremental según el avance.

C.2.2 Riesgos previstos que se materializaron

De los cuatro, la disponibilidad de hardware se controló según lo previsto y no llegó a afectar el trabajo; los otros tres se materializaron, uno con una forma distinta de la prevista.

La complejidad de la implementación distribuida en Rust. Fue el riesgo dominante. Las garantías de *fearless concurrency* cumplieron lo que prometen, ya que ninguna condición de carrera sobre memoria compartida llegó a la rama principal, pero desplazaron la dificultad hacia la coordinación distribuida, que el compilador no puede verificar porque no es un problema de memoria sino de protocolo. La dificultad real fue sincronizar el estado de fase de cada entidad dentro del algoritmo que ejecuta, sobre todo en All-Reduce, donde cada nodo avanza por pasos que dependen de sus vecinos.

La amplitud del alcance y la estimación de tiempos. La mitigación prevista, priorizar un sistema base funcional y optimizar de forma incremental, se aplicó y funcionó. El costo fue que algunas líneas postergadas no llegaron a cerrarse, según se detalla más abajo.

La dependencia de trabajos previos. Se materializó de una forma no anticipada. Lo previsto era que la interpretación de un algoritmo publicado difiriera de lo documentado; lo que ocurrió fue más sutil: los supuestos de los papers no siempre se cumplen en el régimen propio.

C.2.3 Riesgos no previstos

Aplicar un algoritmo sin verificar la validez de sus supuestos. El sistema implementa la actualización de parámetros libre de bloqueos siguiendo a Niu et al. (2011), y una auditoría tardía mostró que la variante no era utilizable. Rust no permite obtener dos múltiples referencias mutables a la misma memoria por defecto, de modo que implementar la técnica del paper exigió utilizar estrategias para sortear esa limitación. En particular, se usó el patrón de *interior mutability* para alojar los parámetros y compartir su mutabilidad. El síntoma fue un cuelgue silencioso que solo aparecía con varios workers, y las pruebas existentes no lo detectaron porque ejercitaban únicamente la variante con bloqueo.

C.2.4 Desvíos de alcance

La optimización de carga en configuraciones heterogéneas no se abordó. Estaba enunciada en la propuesta y el entorno de contenedores admitía representar nodos de capacidades distintas, de modo que no hubo un impedimento técnico: la tarea se fue postergando frente a otras prioridades y no llegó a abordarse.

Las pruebas de integración de alto nivel fueron reemplazadas. Las que levantaban un clúster completo desde Python se abandonaron en favor de la suite de benchmarks: sin un estándar acordado para las configuraciones que requerían, su mantenimiento superaba la confianza que aportaban.

C.2.5 Lecciones aprendidas

Conviene tener una medición *end-to-end* desde temprano. La suite de benchmarks llegó al final y con otro propósito, el de comparar estrategias, y aun así terminó destapando tres errores de corrección que llevaban meses en el código: All-Reduce sumaba los gradientes en lugar de promediarlos, con lo que la tasa de aprendizaje efectiva escalaba con la cantidad de workers; al recuperar de los servidores los pesos entrenados, el orchestrator no revertía el reparto de capas que él mismo había hecho, de modo que el modelo final quedaba con sus capas desordenadas; y esto se observó recién en *Strategy-Switch* cuando al conmutar de estrategia, los fragmentos se asignaban en un orden distinto del que usaba el worker para indexarlos. Los tres eran silenciosos: no provocaban fallos, solo degradaban la exactitud, y ninguna prueba unitaria los habría encontrado porque cada componente hacía correctamente lo suyo.

Lo perfecto es enemigo de lo bueno. Varios tiempos de trabajo sobre la implementación de funcionalidades del sistema difirieron mucho con respecto a lo esperado. La razón fue la "búsqueda de la perfección", una búsqueda cuyo encuentro se vuelve inalcanzable y que es un problema común en proyectos de este estilo. En particular, el desarrollo del motor de redes neuronales ocupó mucho más tiempo del planificado, debido principalmente a intentos de optimizar el tiempo de cómputo. Conviene siempre trabajar en ítems que conformen cortes transversales de funcionalidad del producto, como sugirió Kent Beck: *"make it work, make it right, make it fast"*.

Anexo D

Repositorio y documentación del proyecto

D.1 Repositorio del proyecto

El código fuente, su documentación técnica y el historial completo del desarrollo están alojados en el siguiente repositorio público:

`https://github.com/lminervino18/oxidized-neural-orchestra`

Está organizado como un *workspace* de Cargo, la herramienta de construcción de Rust, y se divide en los siguientes módulos:

- `machine_learning`: el motor de redes neuronales, con las capas, los optimizadores, las funciones de pérdida y la representación de tensores.
- `comms`: la capa de comunicación entre nodos y su protocolo propio.
- `parameter_server`: la implementación de *Parameter Server*.
- `worker`: el runtime del nodo worker, que incluye la implementación en anillo de *All-Reduce*.
- `orchestrator`: el plano de control, que asigna los roles y dirige la ejecución de un entrenamiento.
- `node`: el proceso de cómputo, agnóstico del rol que le sea asignado.
- `orchestra-py`: la biblioteca de Python, construida sobre la interfaz de funciones externas.
- `orchestui`: la interfaz interactiva de terminal.
- `benchmarks`: la suite de experimentación y los generadores de gráficos.
- `docker`: los scripts que levantan los entornos de simulación multinodo.
- `docs`: la propuesta, el presente informe y la documentación de diseño del sistema.

D.2 Documentación técnica

Dentro del repositorio se encuentran los siguientes documentos, referenciados a lo largo de este informe:

- `docs/config-schema.md`: la referencia canónica del esquema de configuración de un entrenamiento, y única fuente de verdad del formato que aceptan las interfaces del sistema.
- `docs/architecture-draft.md`: el borrador de arquitectura que sostuvo las discusiones de diseño de las etapas iniciales.

- `docs/report/roadmap.md`: la reconstrucción del recorrido conceptual del proyecto a partir del historial del repositorio, con las decisiones de arquitectura, los debates que las motivaron y su fundamentación bibliográfica. Es el documento de apoyo del que derivan *Metodología de trabajo aplicada*, *Cronograma* y *Riesgos materializados y lecciones aprendidas*.
- `benchmarks/README.md`: la documentación de la suite de experimentación, con las configuraciones de cada campaña y las instrucciones para reproducirlas.
- Los `README.md` de cada crate, con la documentación de sus abstracciones principales.

D.3 Reproducibilidad de los experimentos

La campaña de benchmarks del sistema es reproducible a partir del material versionado en el repositorio. Se ejecuta mediante `benchmarks/run_issue_benchmarks.py`, cubre 38 configuraciones con tres repeticiones en las suites de throughput y de escalabilidad, que se reportan en media y desvío, y conserva las configuraciones que generaron cada corrida junto con los resultados crudos, los procesados y los generadores de figuras.

El estudio experimental fue desarrollado originalmente como una de las dos monografías individuales que dos de los integrantes del equipo desarrollaron en el marco de la materia Aprendizaje Profundo, sobre un arnés de experimentación propio que no forma parte de este repositorio. Sus configuraciones, sus resultados y el criterio de comparación de cada uno se documentan en *Estudio derivado: impacto de los epochs offline*, y las monografías completas se ponen a disposición como documentos separados.

Las semillas de inicialización de los parámetros están fijadas en todas las configuraciones. Debe tenerse presente, no obstante, que las mediciones de tiempo dependen del hardware y de la carga de la máquina en la que se ejecuten, de modo que la reproducibilidad exacta alcanza a las métricas de convergencia y de exactitud, y no a las de rendimiento.